LUMS

CENTER FOR SPEECH AND LANGUAGE TECHNOLOGIES

Motivating Linearity

Agha Ali Raza

CS535/EE514 – Machine Learning

The Regression Problem

$101,000

$143,010

$417,350

$250,120

$196,666

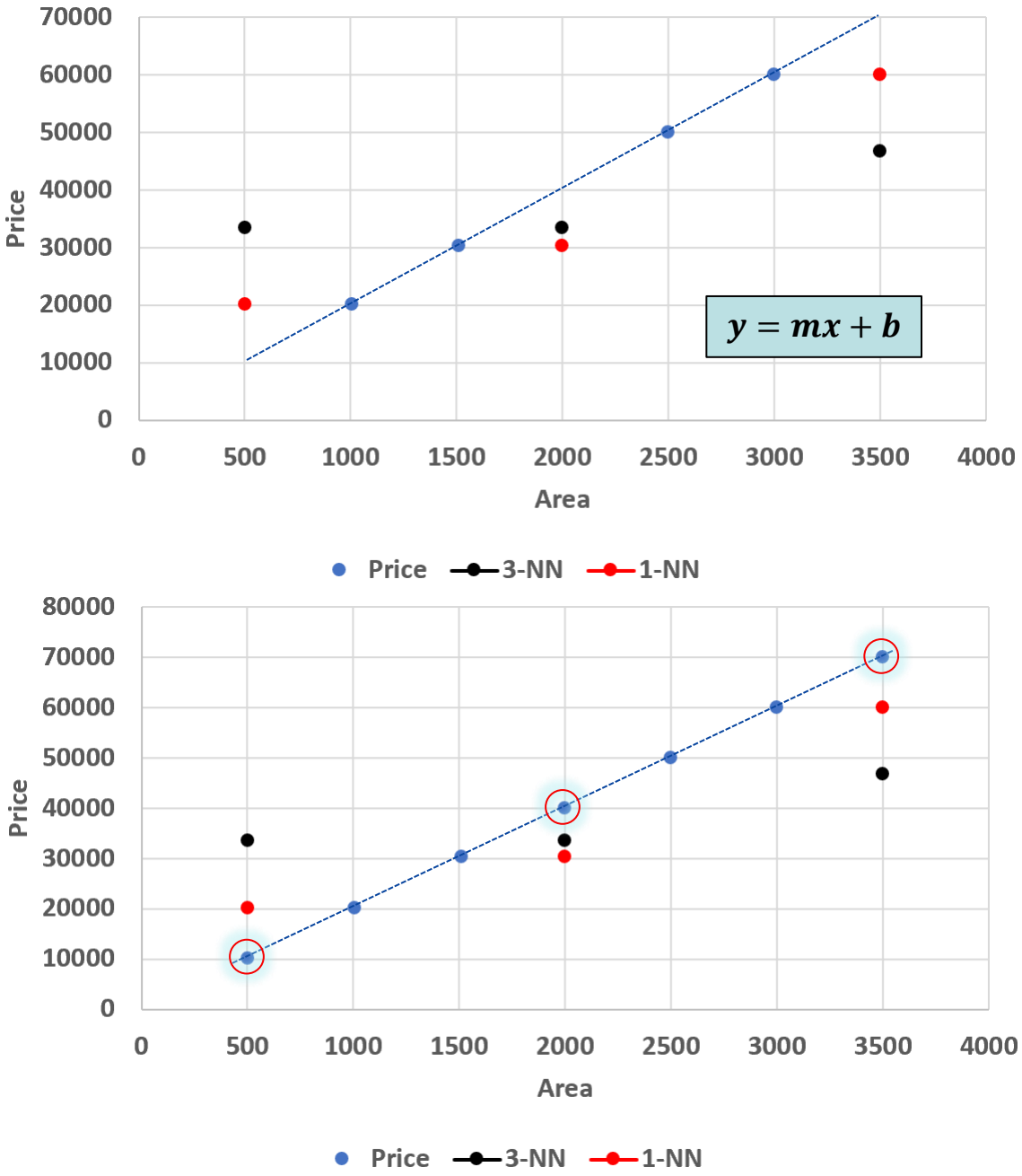
Price?

Find the price!

Area	Price
1,510	30,250
1,005	20,150
2,500	50,050
3,000	60,050

Price for Area: 500? 2000?, 3500?

Area	1-NN	3-NN
500	20,150	30,150
2,000	30,250	33,483.33
3,500	60,050	50,050



LUMS

CENTER FOR SPEECH AND LANGUAGE TECHNOLOGIES

Revision

Lines, planes, hyperplanes and vectors

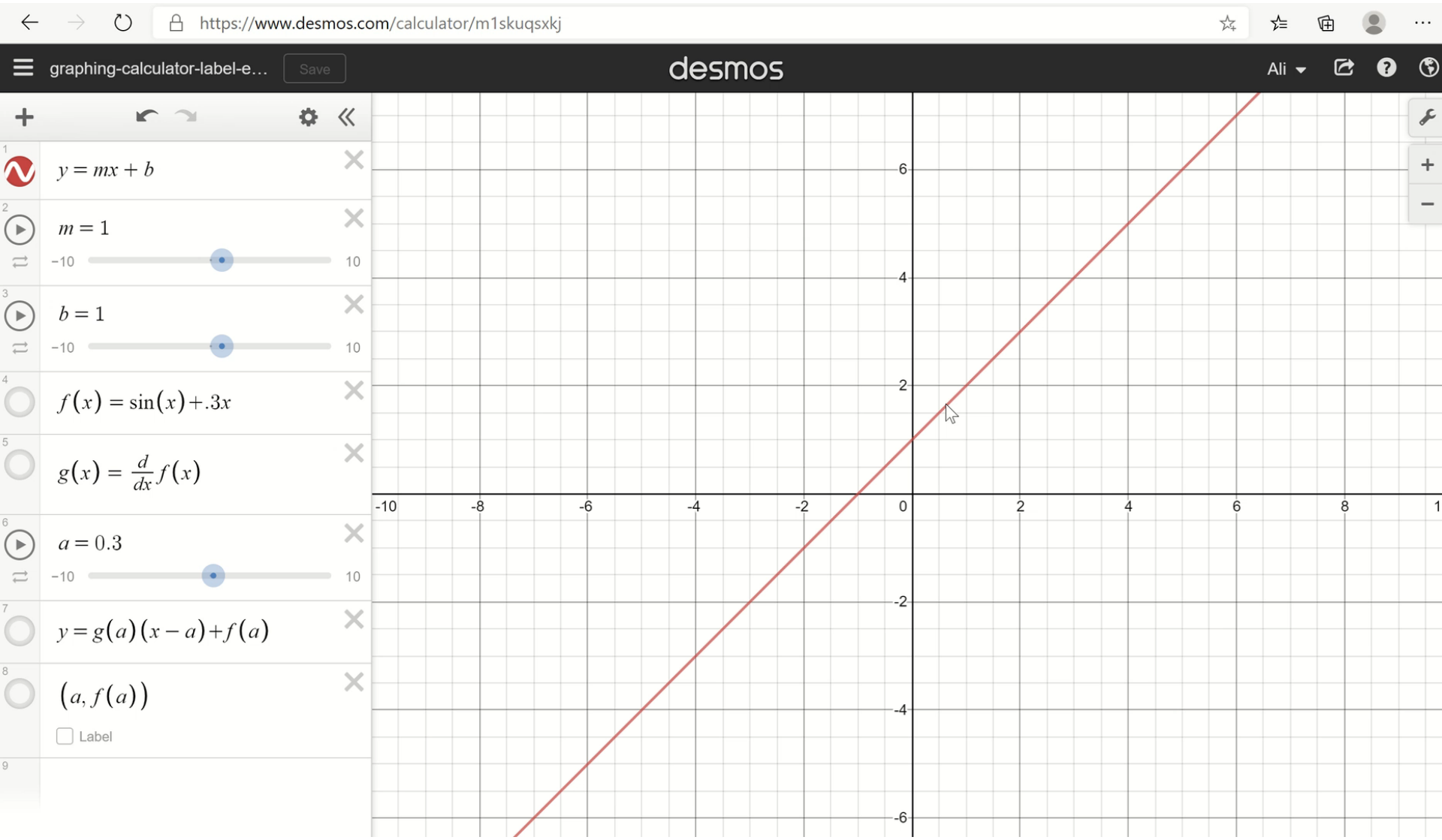
Agha Ali Raza

CS535/EE514 – Machine Learning

Sources

- **Worldwide Calculus: Lines, Planes, and Hyperplanes,** https://www.youtube.com/watch?v=-sNDkhE2Vsk
- **Akshay L Chandra – medium.com**
  - **McCulloch-Pitts Neuron—Mankind's First Mathematical Model Of A Biological Neuron:** https://towardsdatascience.com/mcculloch-pitts-model-5fdf65ac5dd1
  - **Perceptron: The Artificial Neuron (An Essential Upgrade To The McCulloch-Pitts Neuron):** https://towardsdatascience.com/perceptron-the-artificial-neuron-4d8c70d5cc8d
  - **Perceptron Learning Algorithm: A Graphical Explanation Of Why It Works:** https://towardsdatascience.com/perceptron-learning-algorithm-d5db0deab975
- **Prof. Mitesh M. Khapra** (https://www.cse.iitm.ac.in/~miteshk/) on NPTEL's (http://nptel.ac.in/) Deep Learning course (https://onlinecourses.nptel.ac.in/noc18_cs41/preview)
- **Machine Learning for Intelligent Systems**, Kilian Weinberger, Cornell, Lectures 3-6, https://www.cs.cornell.edu/courses/cs4780/2018fa/lectures/lecturenote03.html
- **Perceptrons. An Introduction to Computational Geometry. Marvin Minsky and Seymour Papert. M.I.T. Press, Cambridge, Mass., 1969.** https://science.sciencemag.org/content/165/3895/780

Line


https://www.desmos.com/calculator/m1skuqsxkj
graphing-calculator-label-e...
Save
desmos
Ali
y = mx + b
m = 1
−10
10
b = 1
−10
10
f(x) = sin(x)+.3x
g(x) = d/dx f(x)
a = 0.3
−10
10
y = g(a)(x − a)+f(a)
(a, f(a))
Label
-10
-8
-6
-4
-2
0
2
4
6
8
6
4
2
-2
-4
-6

Line

Line: $\boldsymbol{y = mx + b}$ (slop-intercept form)

$\boldsymbol{ax + by + c = 0}$, where $(a, b) \neq (0,0)$ (standard form of the line)

Another way of looking at this is to suppose that (x_0, y_0) is a point on the line, then we can solve for c as:

$$ax_0 + by_0 + c = 0$$
$$c = -ax_0 - by_0$$

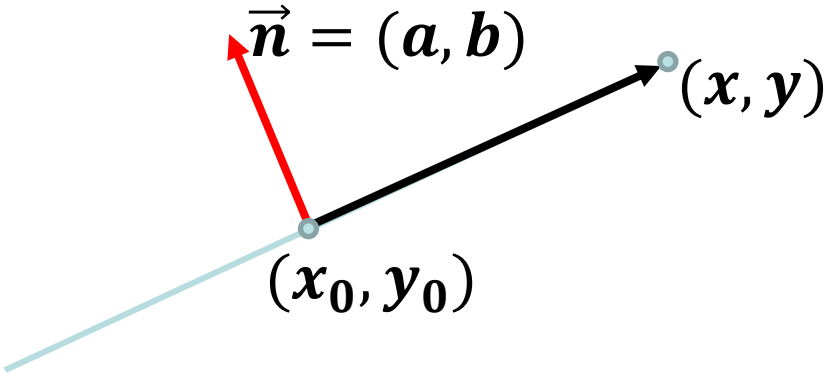
Therefore,

$$ax + by - ax_o - by_0 = 0$$
$$a(x - x_0) + b(y - y_0) = 0$$

That can be written as a dot product of two vectors:

$$(a, b).(x - x_0, y - y_0) = 0$$

But, when the dot product is 0 and $(a, b) \neq (0,0)$, the two vectors must be perpendicular.



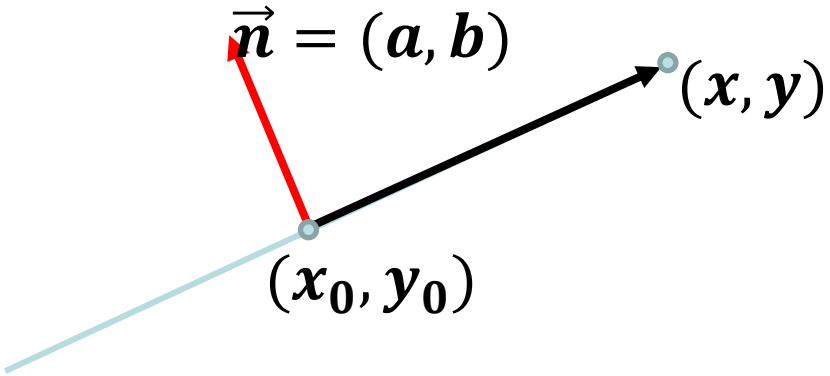
Line

Let $\vec{n} = (a, b) \neq 0$, then

$$(\boldsymbol{a}, \boldsymbol{b}).(\boldsymbol{x} - \boldsymbol{x_0}, \boldsymbol{y} - \boldsymbol{y_0}) = \boldsymbol{0}$$

describes a line containing the point (x_0, y_0), perpendicular to the vector $\vec{n}$.

This is equivalent to the standard form of the line $\boldsymbol{ax} + \boldsymbol{by} + \boldsymbol{c} = \boldsymbol{0}$, where $(a, b) \neq (0,0)$.



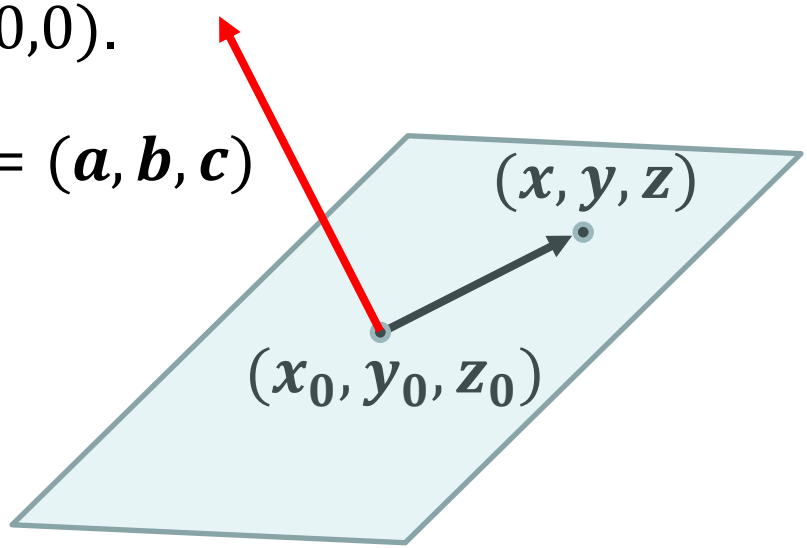
Plane

We can extend the same notion to describe a plane, using a normal vector $\vec{n} = (a, b, c) \neq 0$, and a point (x_0, y_0, z_0) on the plane:

$$(\boldsymbol{a}, \boldsymbol{b}, \boldsymbol{c}).(\boldsymbol{x} - \boldsymbol{x_0}, \boldsymbol{y} - \boldsymbol{y_0}, \boldsymbol{z} - \boldsymbol{z_0}) = \boldsymbol{0}$$

describes a plane containing the point (x_0, y_0, z_0), perpendicular to the vector $\vec{n}$.

This is equivalent to the standard form of the plane $\boldsymbol{ax} + \boldsymbol{by} + \boldsymbol{cz} + \boldsymbol{d} = \boldsymbol{0}$, where $(a, b, c) \neq (0,0)$.



Decomposition

Hyperplane

A hyperplane is a subspace whose dimension is one less than that of its ambient space

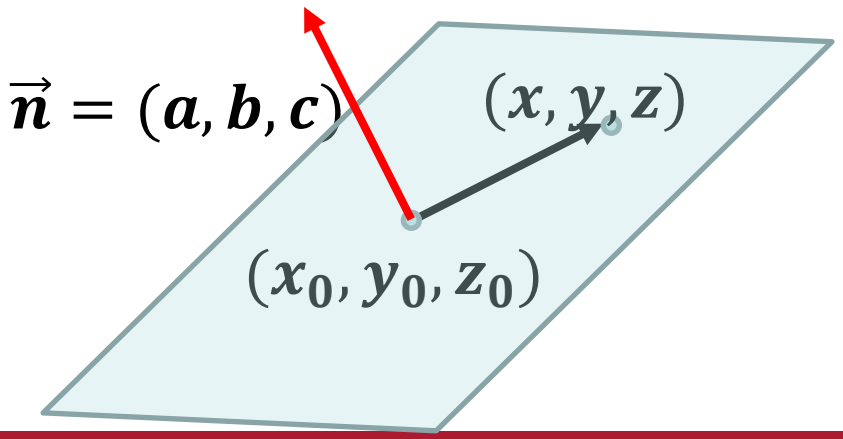
- If a space is 3-dimensional then its hyperplanes are the 2-dimensional planes
- If the space is 2-dimensional, its hyperplanes are the 1-dimensional lines

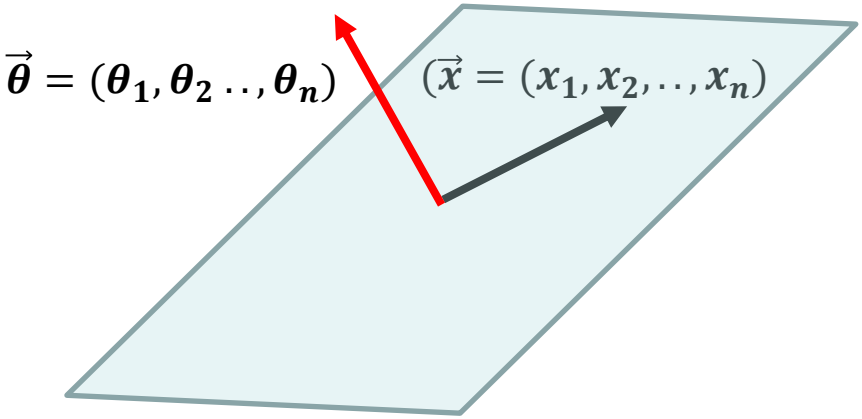
$$a_1x_1 + a_2x_2 + \cdots + a_nx_n = b$$

When the coordinates are real numbers, this hyperplane **separates the space into two half-spaces** given by the inequalities

$$a_1x_1 + a_2x_2 + \cdots + a_nx_n < b$$

$$a_1x_1 + a_2x_2 + \cdots + a_nx_n \geq b$$





Distance between a Hyperplane and a Point

Given a hyperplane that passes through a point P and with unit normal vector $\vec{w}$, and a point X, the shortest distance between the point X, and the hyperplane is given as:

$$\left\|\vec{w}.\overrightarrow{PX}\right\|$$

- We can position the normal to any point on the hyperplane, closest to the point x.
- To find the distance we need to define a vector $\overrightarrow{PX}$ originating from a point P on the hyperplane

$$\overrightarrow{PX} = ((x_1 - p_1), (x_2 - p_2), \dots, (x_n - p_n))$$

- The shortest distance from the point X to the hyperplane is then just the projection of $\overrightarrow{PX}$ on $\vec{w}$, $\overrightarrow{PX}\cos\theta$, where θ is the angle between $\overrightarrow{PX}$ and $\vec{w}$.

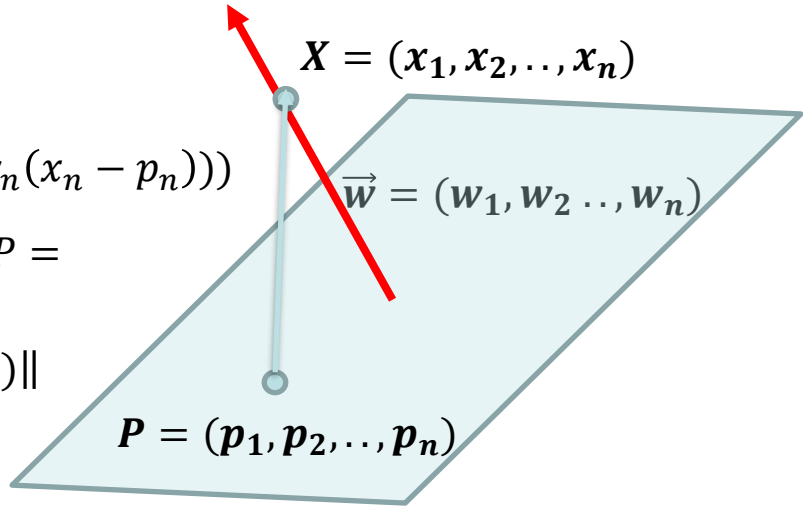
$$\vec{w}.\overrightarrow{PX} = \|\vec{w}\| \left\|\overrightarrow{PX}\right\| cos\theta$$

- As $\vec{w}$ is a unit vector, $\|\vec{w}\| = 1$

$$\left\|\overrightarrow{PX}\right\| cos\theta = \vec{w}.\overrightarrow{PX} = (w_1(x_1 - p_1)), (w_2(x_2 - p_2)), \dots, (w_n(x_n - p_n)))$$

- If we know that the hyperplane passes through the origin, $P = (0,0, \dots, 0)$, we just find the magnitude of:

$$\left\|\vec{w}.\overrightarrow{PX}\right\| = \left\|\vec{w}.\vec{X}\right\| = \|(w_1x_1), (w_2x_2), \dots, (w_nx_n)\|$$



The Dot Product is Commutative

For two vectors, $\boldsymbol{A}$ and $\boldsymbol{B}$

$$\boldsymbol{A}.\boldsymbol{B} = \boldsymbol{B}.\boldsymbol{A}$$

As

$$\boldsymbol{A}.\boldsymbol{B} = A^T B$$

And

$$\boldsymbol{B}.\boldsymbol{A} = B^T A$$

Therefore,

$$A^T B = B^T A$$

Geometric Interpretation of absorbing the Bias

(Kilian Weinberger, Lecture 3: The Perceptron, https://www.cs.cornell.edu/courses/cs4780/2018fa/lectures/lecturenote03.html)

For mathematical convenience we often convert from the following form of the linear expression:

$$\boldsymbol{a_1 x_1 + a_2 x_2 + \cdots + a_n x_n - b = 0}$$

To

$$\boldsymbol{a_0 x_0 + a_1 x_1 + a_2 x_2 + \cdots + a_n x_n = 0}$$

Where $x_0 = 1$ and $a_0 = -b$, and the number of dimensions have been increased to $n + 1$.

This allows us to just bundle the bias (the intercept) into the expression and represent everything as:

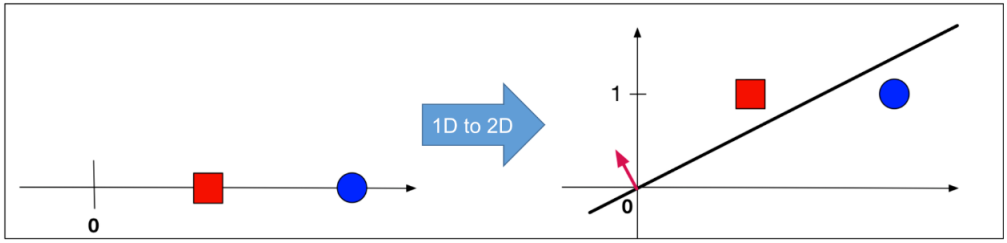
$$\sum_{i=0}^{n} a_i x_i = \vec{a}.\vec{x} = a^T x$$

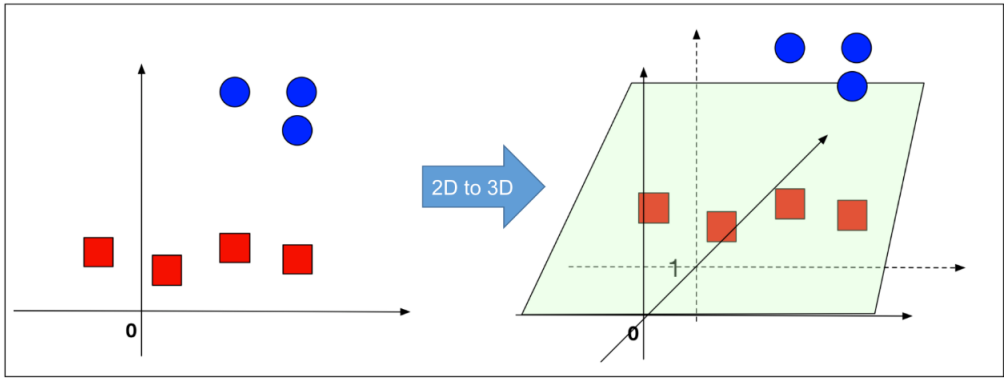
But this also has a very important geometric interpretation as well.

As

$$\sum_{i=0}^{n} a_i x_i + 0 = 0$$

We can claim that this new n+1 dimensional hyperplane always passes through the origin, but still, the old n-dimensional action takes places at $x_0 = 1$.





(Left:) The original data is 1-dimensional (top row) or 2-dimensional (bottom row). There is no hyper-plane that passes through the origin and separates the red and blue points. (Right:) After a constant dimension was added to all data points such a hyperplane exists.

Credit: **Kilian Weinberger, Lecture 3: The Perceptron**

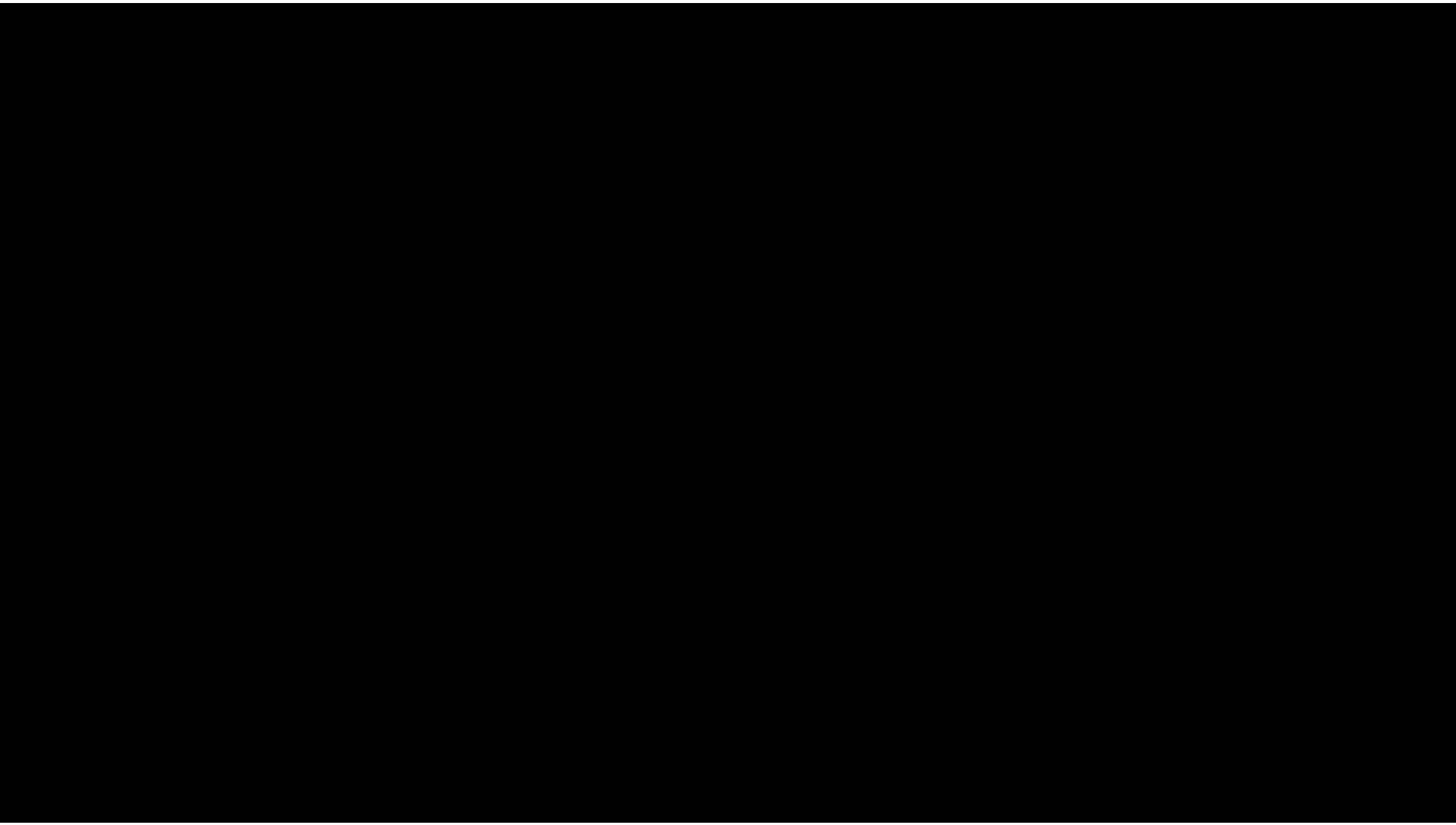
Visualizing n-dimensions

How to imagine the tenth dimension:
https://www.youtube.com/watch?v=0ca4miMMaCE&list=PLnvLVSNZy9VIlt7T-DilHj_65uW3br4sX&index=169

Visualizing n-dimensions

Möbius Strip | Klein Bottle | A Mind-Blowing 4D Visualization: https://www.youtube.com/watch?v=JmvHNatZgVI

Visualizing n-dimensions



TEDxBoulder - Thad Roberts - Visualizing Eleven Dimensions: https://www.youtube.com/watch?v=aSz5BjExs9o

Visualizing n-dimensions

Drawing the 4th, 5th, 6th, and 7th dimension - Visualizing Eleven Dimensions: https://www.youtube.com/watch?v=Q_B5GpsbSQw

Two Möbius strips to form a Klein bottle

https://en.wikipedia.org/wiki/M%C3%B6bius_strip

LUMS

CENTER FOR SPEECH AND LANGUAGE TECHNOLOGIES

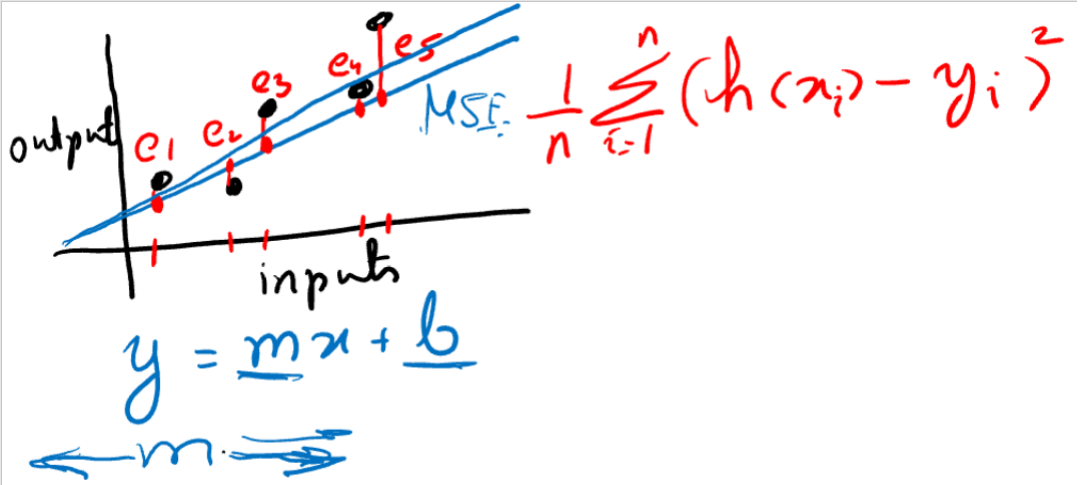
Linear Regression
Intuition

Agha Ali Raza

CS535/EE514 – Machine Learning

A quick overview of it all

- Fitting a line to data
- How to predict using the equation of line
- How to fit the line to data?
  - Cost function: Mean Square Error and OLS
  - Plot of the cost function
  - Gradients of the cost function
  - Stepping down the slopes: Gradient descent
  - Pulling the strings one-by-one, all-at-once or some-at-once
  - Convex and non-convex cost functions
  - Local and global minima


output
inputs
e_1
e_2
e_3
e_4
e_5
MSE $\frac{1}{n}\sum_{i=1}^{n}(h(x_i)-y_i)^2$
$y = mx + b$
m

LUMS

CENTER FOR SPEECH AND LANGUAGE TECHNOLOGIES

Linear Regression
Setup

Agha Ali Raza

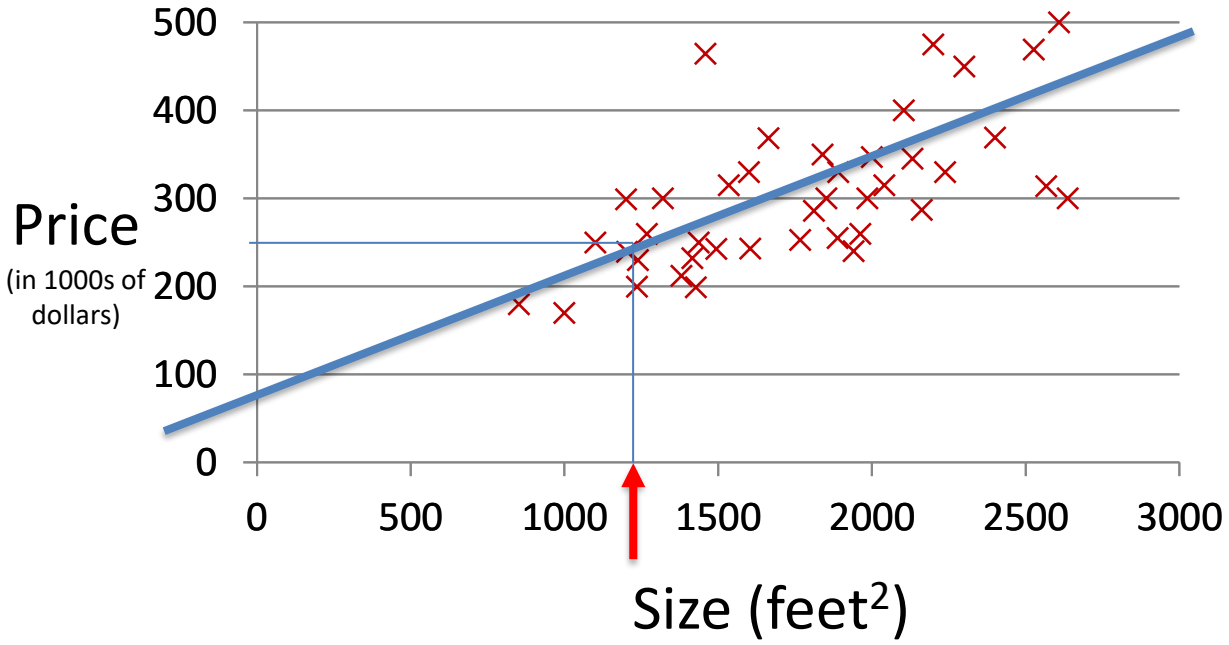
CS535/EE514 – Machine Learning

References

- Machine Learning, Andrew Ng, on Coursera by Stanford – a https://www.coursera.org/learn/machine-learning
- Deep Learning Specialization, Andrew Ng, on Coursera by deeplearning.ai – https://www.coursera.org/specializations/deep-learning

Linear Regression with one variable

Size ($feet^2$)	Price \$(×1000)
1500	190
2250	285
2740	420
2318	300
2500	350
1250	180
…	…



Notation:

m = Number of training samples

n = Number of features

x_j = jth feature

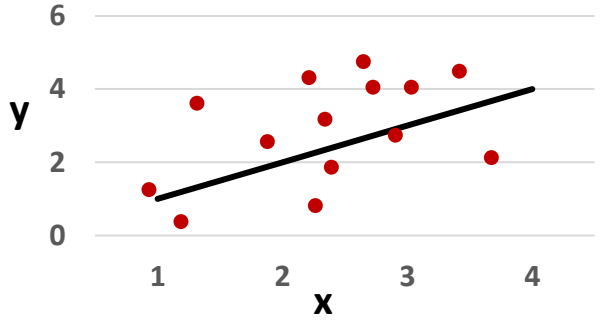
y = Label

(x^i, y^i): the ith sample in the dataset

Linear Regression with one variable

$$h_\theta(x) = \theta_0 + \theta_1 x$$

Linear regression with one variable, univariate linear regression, simple linear regression



Choose θ_0, θ_1 such that $h_\theta(x) \approx y$ for our training examples (x, y).

Parameters

$$\theta_0, \theta_1$$

Cost function

$$J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^{m} \left(h_\theta\left(x^{(i)}\right) - y^{(i)} \right)^2$$

Goal:

$$Minimize_{\theta_0, \theta_1} J(\theta_0, \theta_1)$$

A simplified case

Hypothesis

$$h_\theta(x) = \theta_0 + \theta_1 x$$

Parameters

$$\theta_0, \theta_1$$

Cost function

$$J(\theta_0, \theta_1) = \frac{1}{2m}\sum_{i=1}^{m}\left(h_\theta\left(x^{(i)}\right) - y^{(i)}\right)^2$$

Goal:

$$Minimize_{\theta_0,\theta_1} J(\theta_0, \theta_1)$$

Assume $\theta_0 = 0$

Hypothesis

$$h_\theta(x) = \theta_1 x$$

Parameters

$$\theta_1$$

Cost function

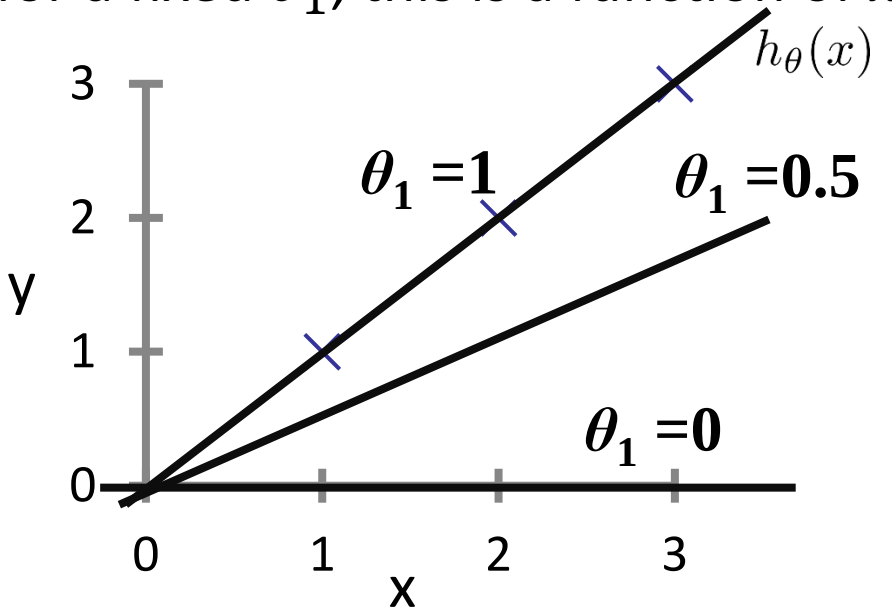
$$J(\theta_1) = \frac{1}{2m}\sum_{i=1}^{m}\left(h_\theta\left(x^{(i)}\right) - y^{(i)}\right)^2$$

Goal:

$$Minimize_{\theta_1} J(\theta_1)$$

$h_\theta(x) = \theta_1 x$

(for a fixed θ_1, this is a function of x)



$$J(\theta_1) = \frac{1}{2m}\sum_{i=1}^{m}(h_\theta(x^{(i)}) - y^{(i)})^2$$

$$J(\theta_1) = \frac{1}{2m}\sum_{i=1}^{m}(\theta_1 x^{(i)} - y^{(i)})^2$$

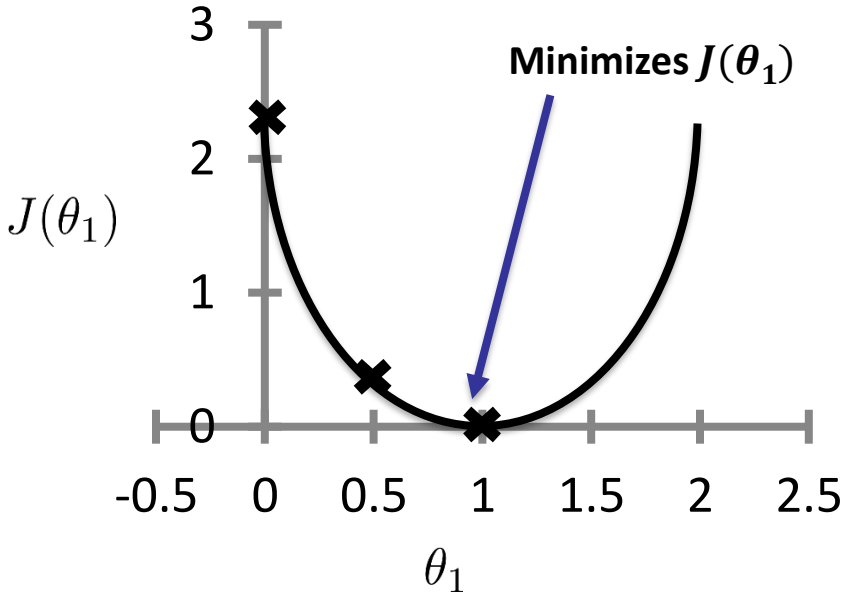
$J(\theta_1)$

(function of the parameter θ_1)

$$J(1) = \frac{1}{2(3)}\left((1-1)^2 + (2-2)^2 + (3-3)^2\right) = 0$$

$$J(0.5) = \frac{1}{6}\left((0.5-1)^2 + (1-2)^2 + (1.5-3)^2\right) = 0.58$$

$$J(0) = \frac{1}{6}\left((1)^2 + (2)^2 + (3)^2\right) = 2.3$$



Using both of the "knobs"

Hypothesis

$$h_\theta(x) = \theta_0 + \theta_1 x$$

Parameters

$$\theta_0, \theta_1$$

Cost function

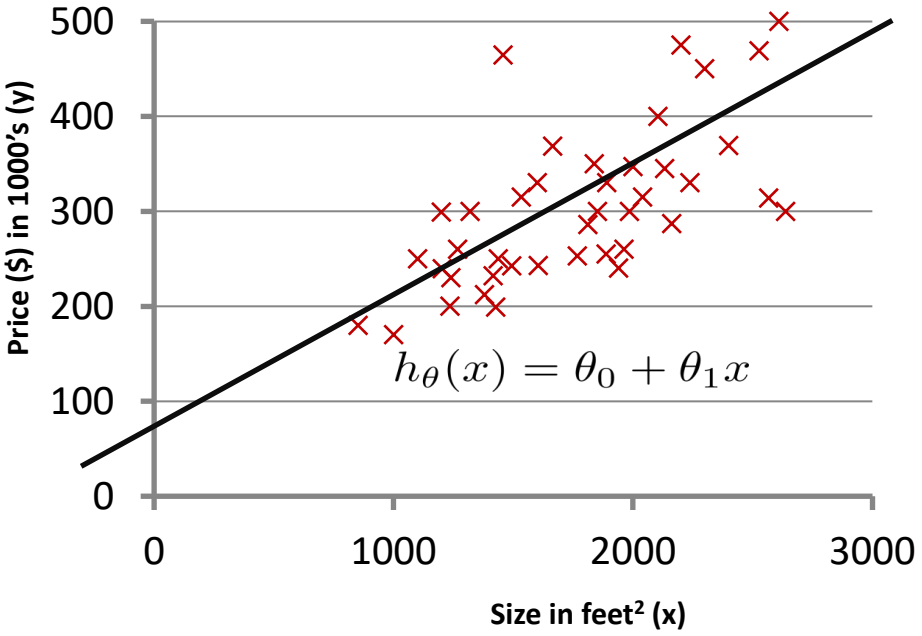
$$J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^{m} \left(h_\theta\left(x^{(i)}\right) - y^{(i)} \right)^2$$

Goal:

$$Minimize_{\theta_0, \theta_1} J(\theta_0, \theta_1)$$

$$h_\theta(x) = \theta_0 + \theta_1 x$$

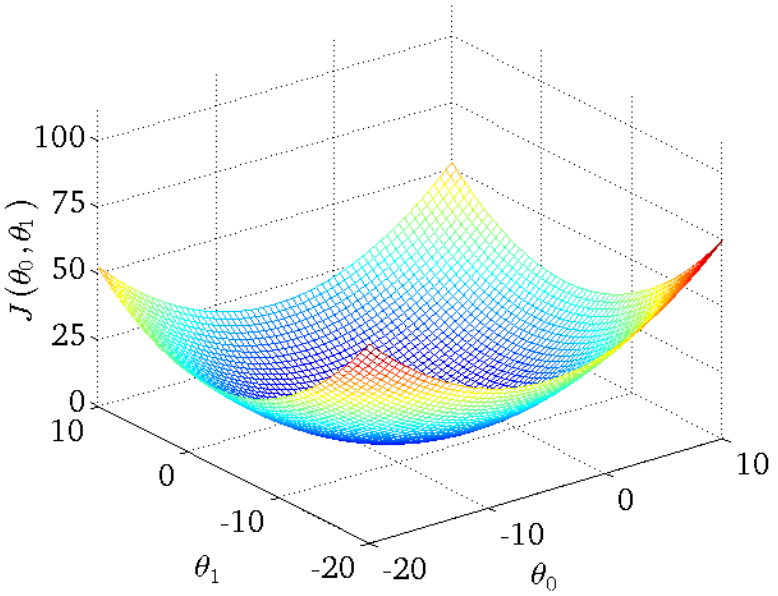
(for fixed θ_0, θ_1, this is a function of x)



$$J(\theta_0, \theta_1)$$

(function of the parameters θ_0, θ_1)

$$J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^{m} \left(h_\theta\left(x^{(i)}\right) - y^{(i)} \right)^2$$

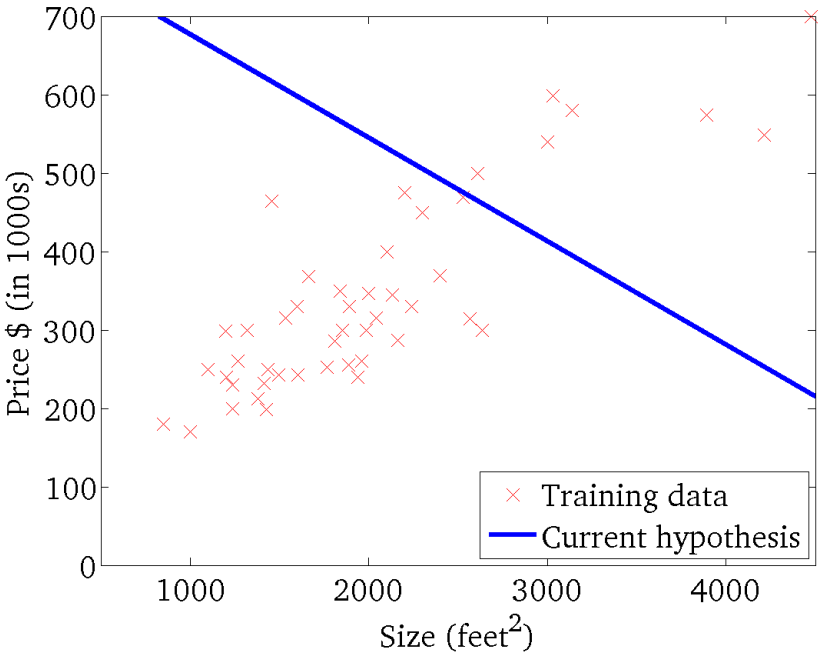


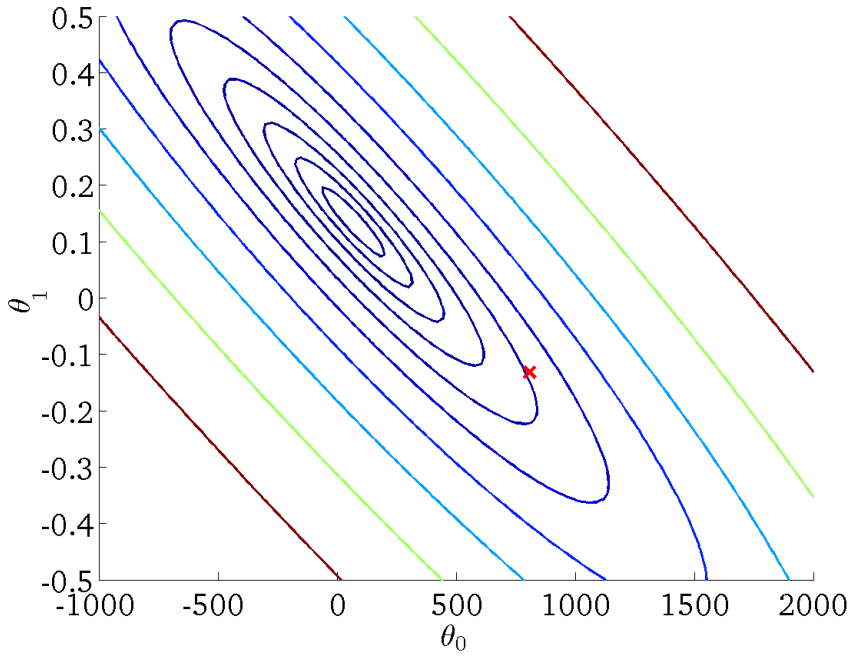
$$h_\theta(x) = \theta_0 + \theta_1 x$$

(for fixed θ_0, θ_1, this is a function of x)

$$J(\theta_0, \theta_1)$$

(function of the parameters θ_0, θ_1)





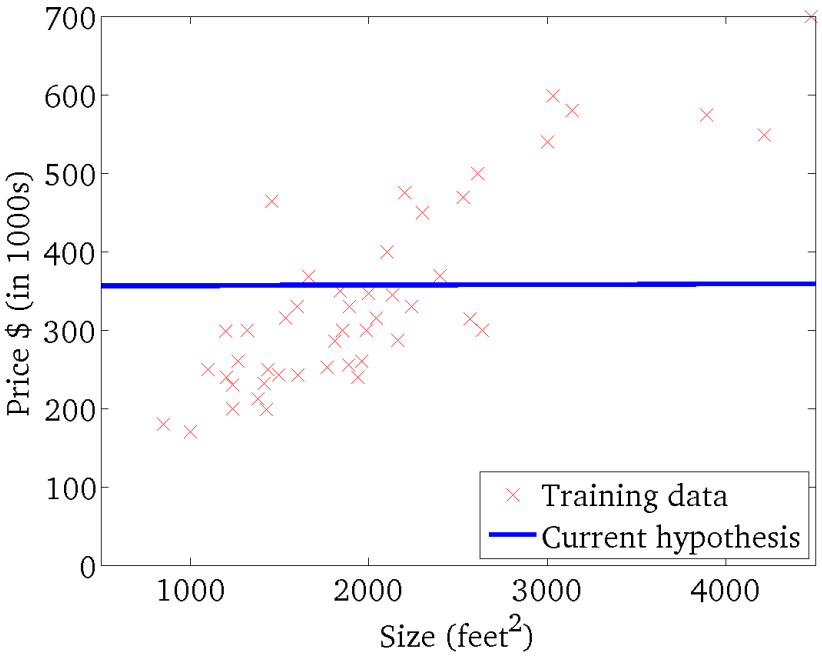
Andrew Ng

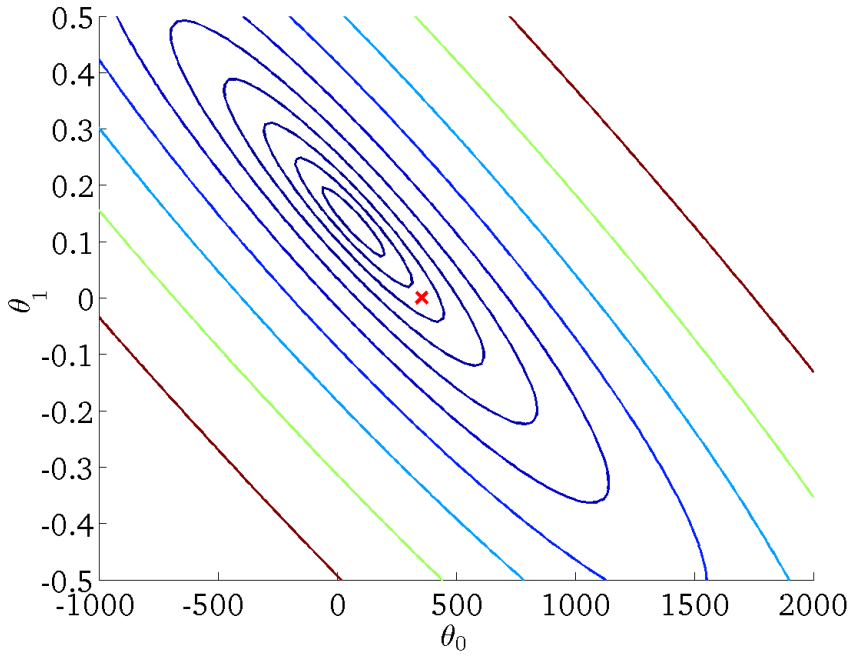
$$h_\theta(x) = \theta_0 + \theta_1 x$$

(for fixed θ_0, θ_1, this is a function of x)

$$J(\theta_0, \theta_1)$$

(function of the parameters θ_0, θ_1)





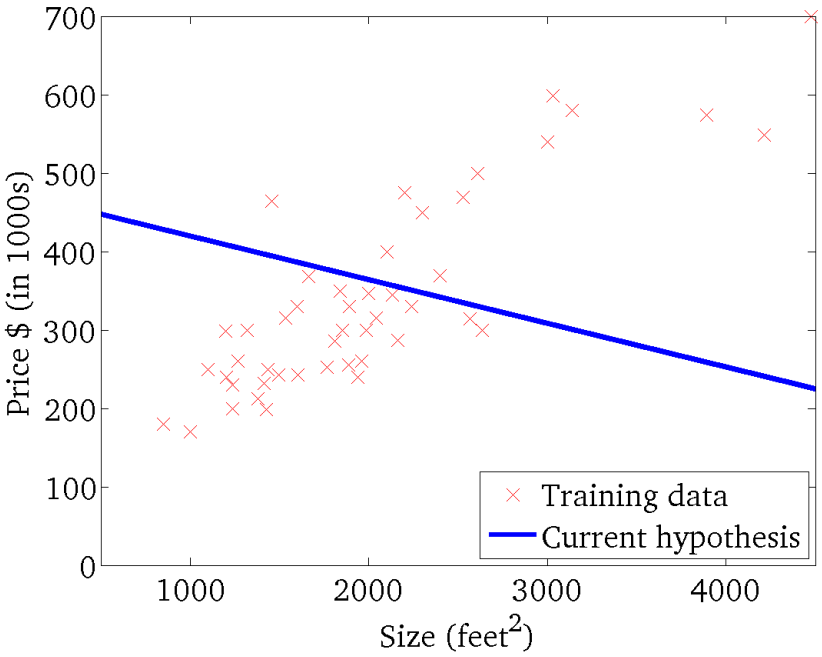
Andrew Ng

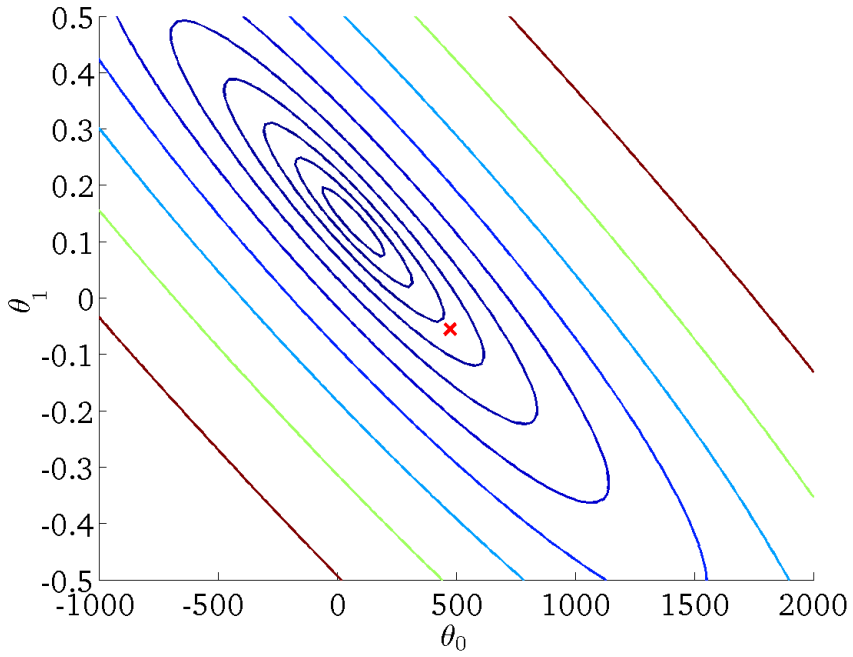
$$h_\theta(x) = \theta_0 + \theta_1 x$$

(for fixed θ_0, θ_1, this is a function of x)

$$J(\theta_0, \theta_1)$$

(function of the parameters θ_0, θ_1)





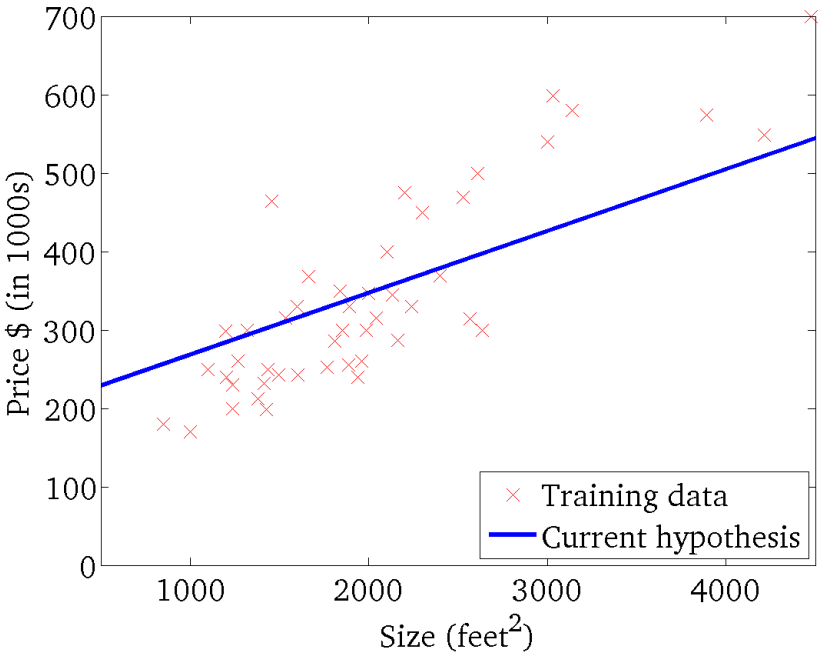
Andrew Ng

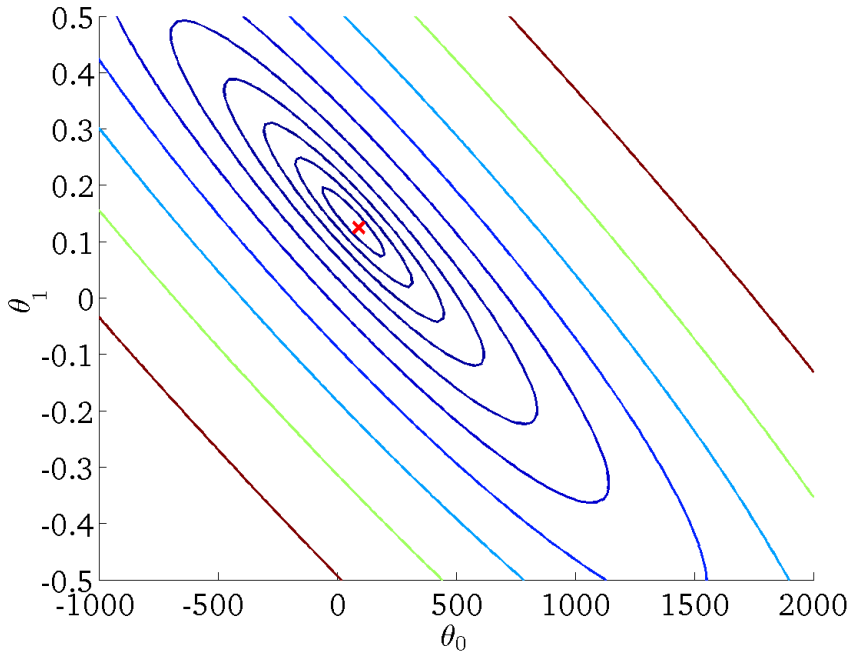
$$h_\theta(x) = \theta_0 + \theta_1 x$$

(for fixed θ_0, θ_1, this is a function of x)

$$J(\theta_0, \theta_1)$$

(function of the parameters θ_0, θ_1)





Andrew Ng

LUMS

CENTER FOR SPEECH AND LANGUAGE TECHNOLOGIES

Linear Regression
The Gradient Descent Algorithm

Agha Ali Raza

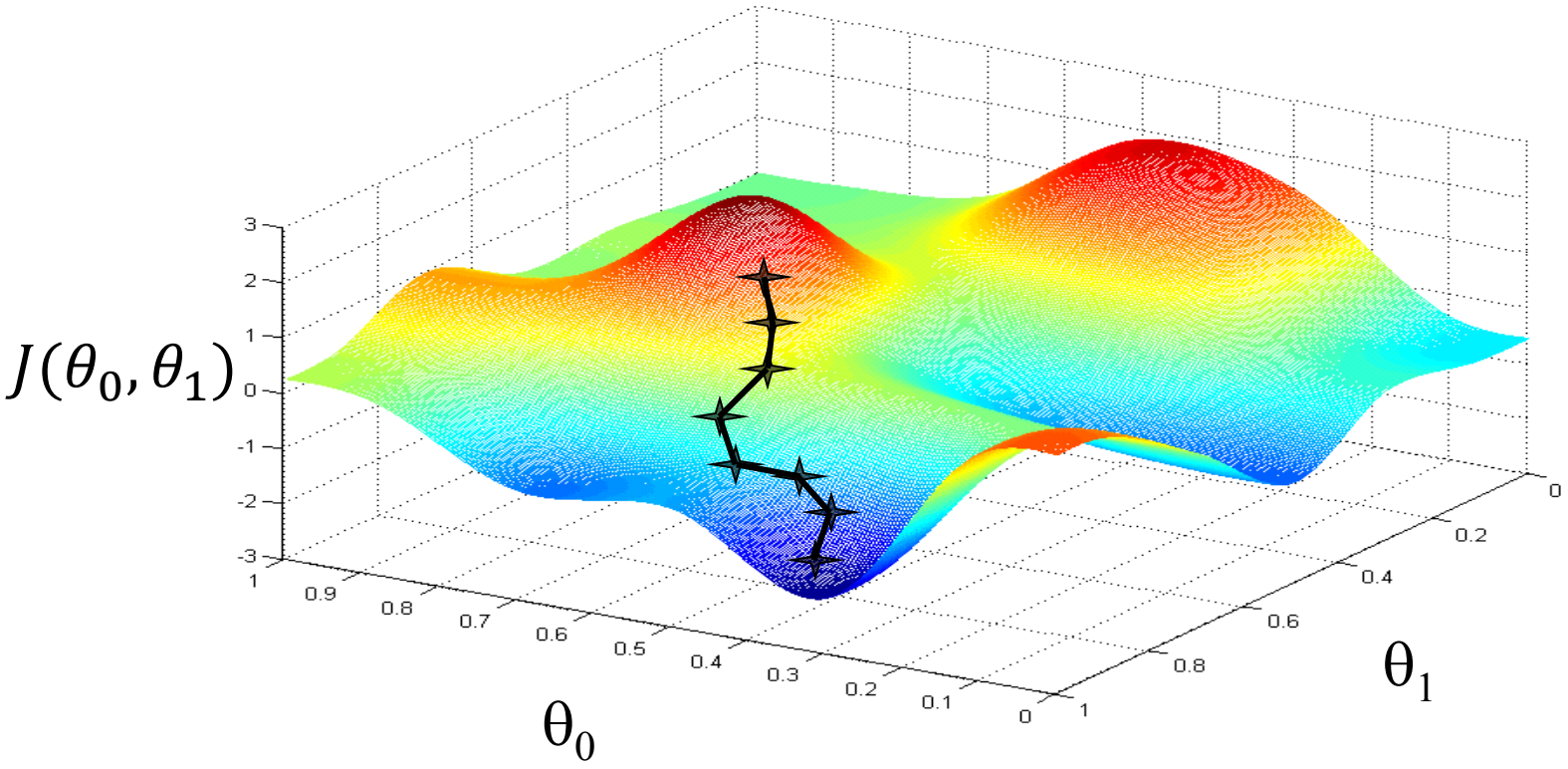
CS535/EE514 – Machine Learning

The Gradient Descent Algorithm

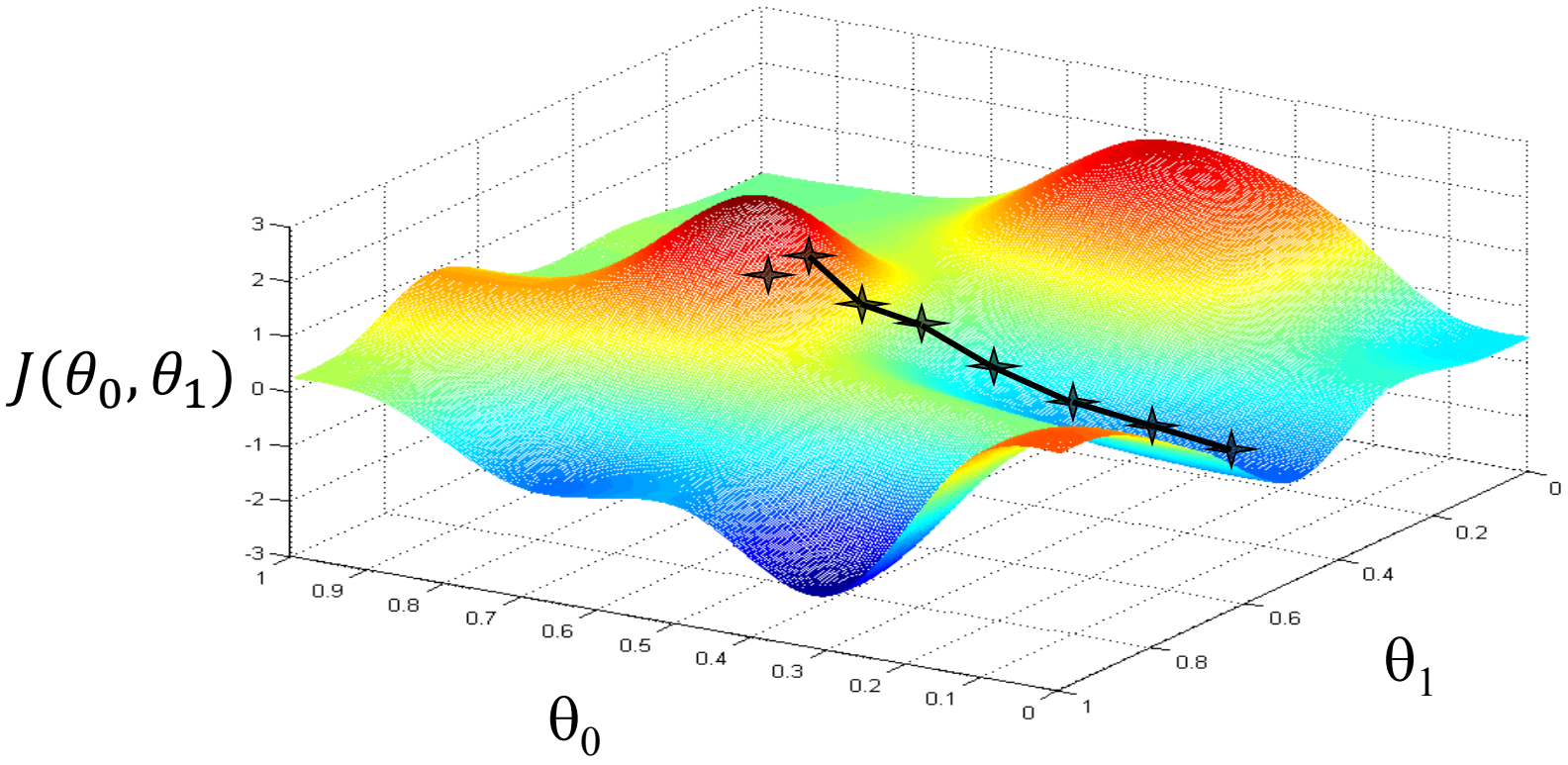
Goal: Minimize $J(\theta_0, \theta_1)$

Outline:

- Start with some (θ_0, θ_1) - For example (0,0)
- Keep updating (θ_0, θ_1) to reduce $J(\theta_0, \theta_1)$
  - Until we **hopefully** reach **a** minimum


$J(\theta_0, \theta_1)$
3
2
1
0
-1
-2
-3
1
0.9
0.8
0.7
0.6
0.5
0.4
0.3
0.2
0.1
0
θ_0
0
0.2
0.4
0.6
0.8
1
θ_1

Andrew Ng


$J(\theta_0, \theta_1)$
3
2
1
0
-1
-2
-3
1
0.9
0.8
0.7
0.6
0.5
0.4
0.3
0.2
0.1
0
θ_0
0
0.2
0.4
0.6
0.8
1
θ_1

Andrew Ng

A simplified version of gradient descent

Assume again that we set $\theta_0 = 0$ and our hypothesis and cost function practically have only one coefficient,θ_1.

$$h_{\theta(x)} = \theta_1 x$$

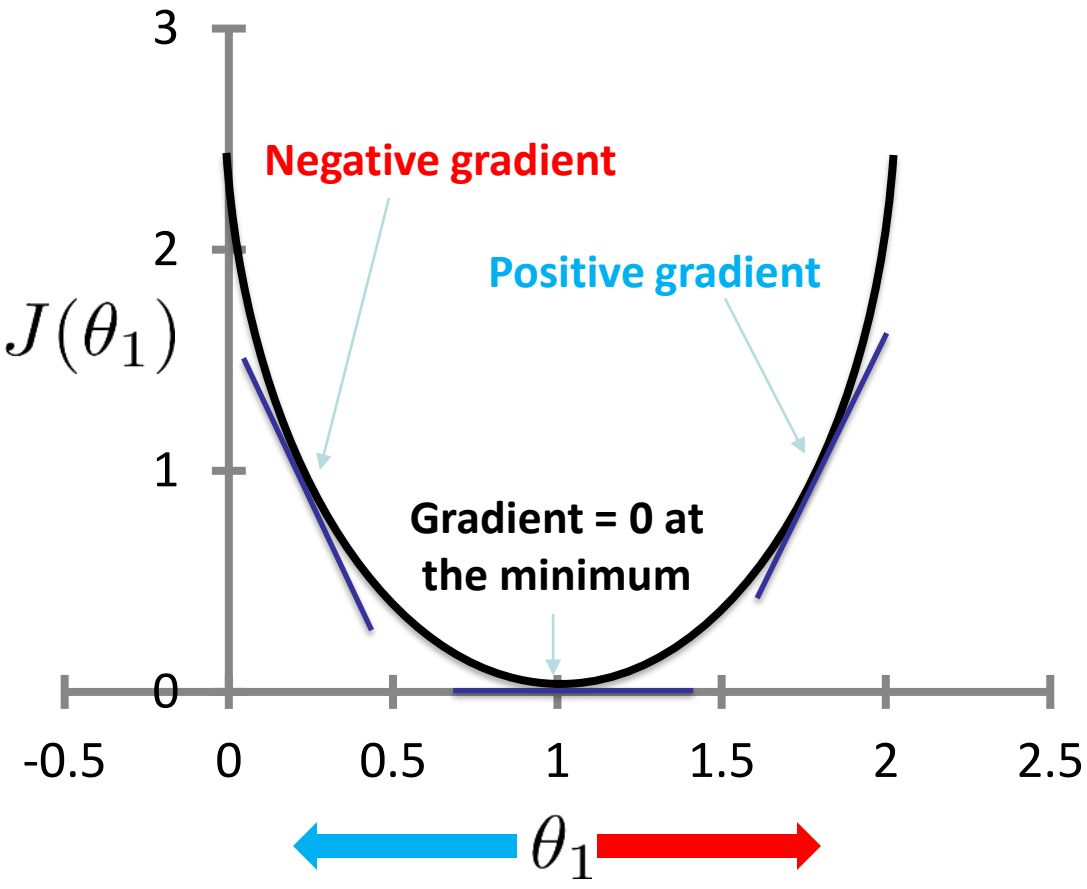
$$repeat\ until\ convergence\{$$

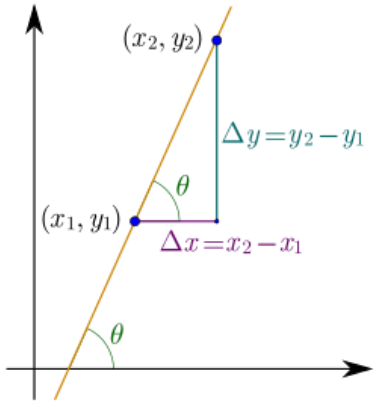
$$\theta_1 := \theta_1 - \alpha \frac{d}{d\theta_1}(J(\theta_1))$$

$$\}$$

$$where\ J(\theta_1) = \frac{1}{2m}\sum_{i=1}^{m}(\theta_1 x^{(i)} - y^{(i)})^2$$

Direction of step





Gradient = $\frac{y_2 - y_1}{x_2 - x_1} = \frac{f(x_2) - f(x_1)}{x_2 - x_1}$

The limiting case is when x_2 approaches x_1

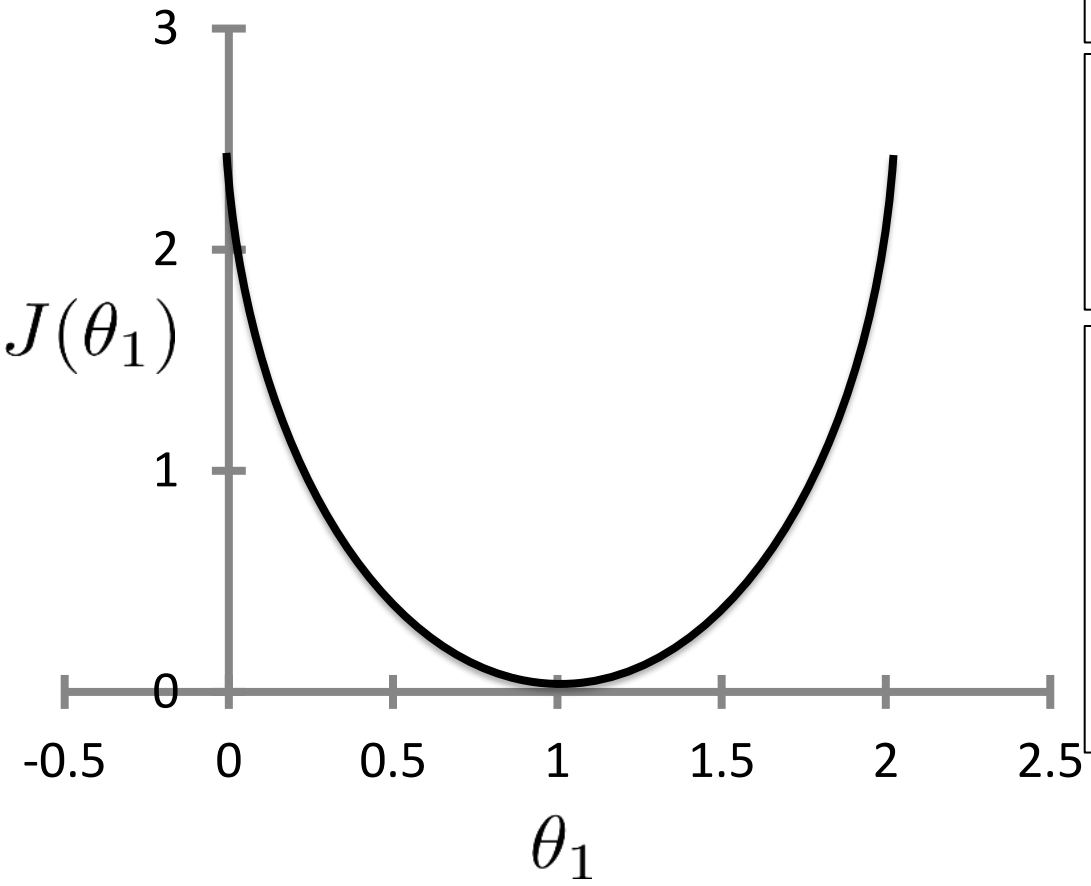
$= \frac{\Delta y}{\Delta x} as \Delta x \rightarrow 0 = \frac{dy}{dx}$

$$\theta_1 := \theta_1 - \alpha \frac{d}{d\theta_1}\left(J(\theta_1)\right)$$

$\theta_1 := \theta_1 - \alpha(negative)$: Increases θ_1

$\theta_1 := \theta_1 - \alpha(positive)$: Decreases θ_1

Step size



$$\theta_1 := \theta_1 - \alpha \frac{\partial}{\partial \theta_1} J(\theta_1)$$

Gradient = $\frac{y_2 - y_1}{x_2 - x_1} = \frac{f(x_2) - f(x_1)}{x_2 - x_1}$

The limiting case is when x_2 approaches x_1

$= \frac{\Delta y}{\Delta x} as \Delta x \rightarrow 0 = \frac{dy}{dx}$

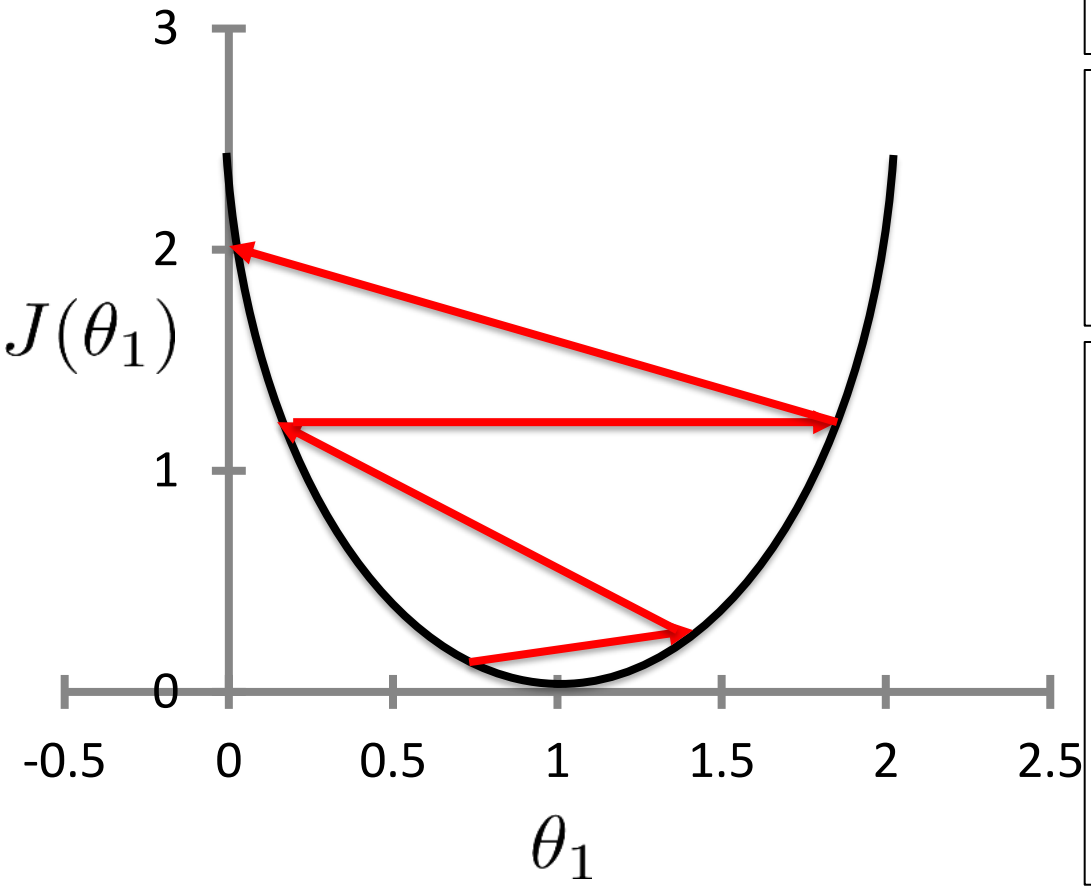
$$\theta_1 := \theta_1 - \alpha \frac{d}{d\theta_1} \left(J(\theta_1)\right)$$

$\theta_1 := \theta_1 - \alpha(negative)$: Increases θ_1

$\theta_1 := \theta_1 - \alpha(positive)$: Decreases θ_1

- In addition to α, the steepness of the gradient also control the step size.
- The step sizes become smaller as we get closer to the minimum, even with a fixed α.
- With α too small, takes a long time to reach the minimum

Step size



$$\theta_1 := \theta_1 - \alpha \frac{\partial}{\partial \theta_1} J(\theta_1)$$

Gradient = $\frac{y_2 - y_1}{x_2 - x_1} = \frac{f(x_2) - f(x_1)}{x_2 - x_1}$

The limiting case is when x_2 approaches x_1

$= \frac{\Delta y}{\Delta x} as \Delta x \rightarrow 0 = \frac{dy}{dx}$

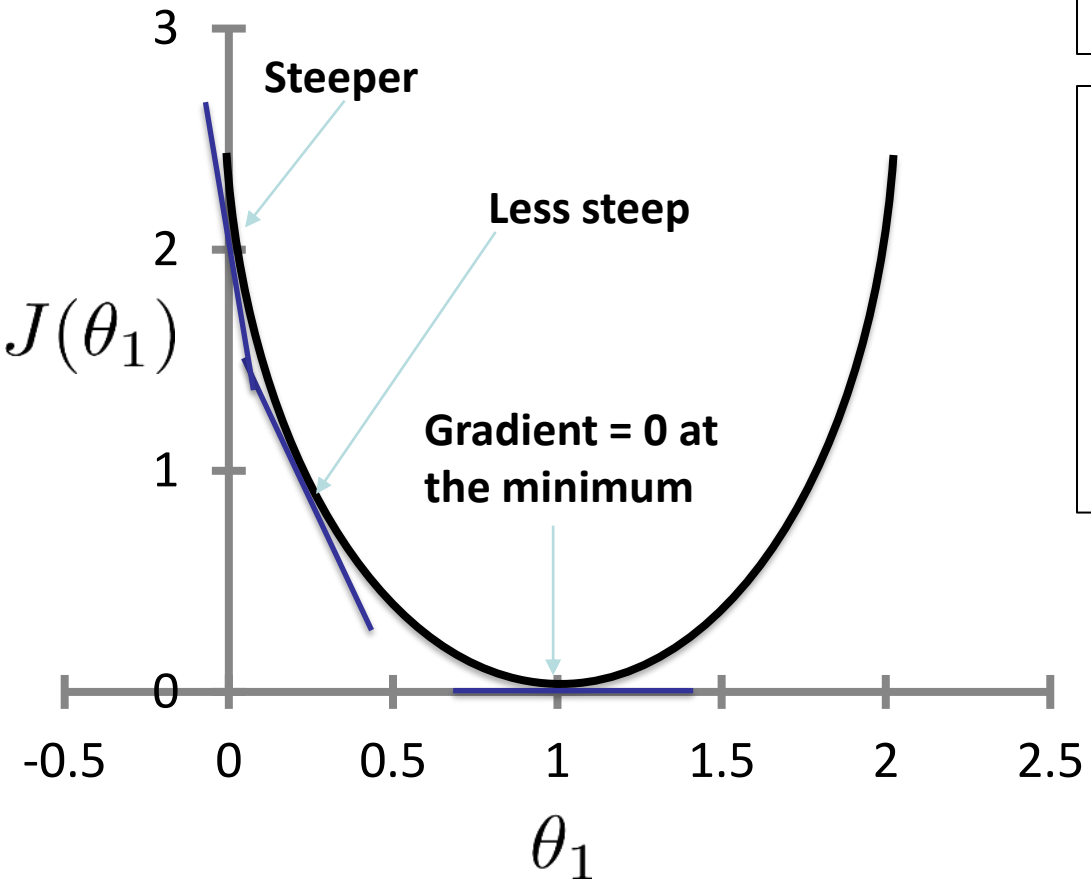
$$\theta_1 := \theta_1 - \alpha \frac{d}{d\theta_1} \left(J(\theta_1)\right)$$

$\theta_1 := \theta_1 - \alpha(negative)$: Increases θ_1

$\theta_1 := \theta_1 - \alpha(positive)$: Decreases θ_1

- In addition to α, the steepness of the gradient also control the step size.
- The step sizes become smaller as we get closer to the minimum, even with a fixed α.
- With α too small, takes a long time to reach the minimum
- With α too big, we can miss the minimum, and may fail to converge

Step size



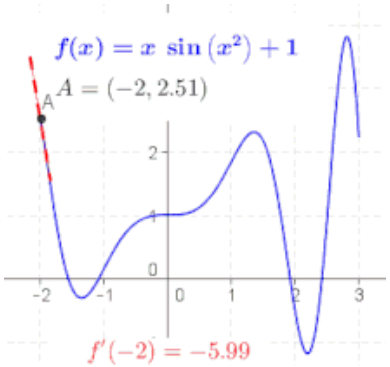
$$\theta_1 := \theta_1 - \alpha \frac{\partial}{\partial \theta_1} J(\theta_1)$$

Gradient = $\frac{y_2 - y_1}{x_2 - x_1} = \frac{f(x_2) - f(x_1)}{x_2 - x_1}$

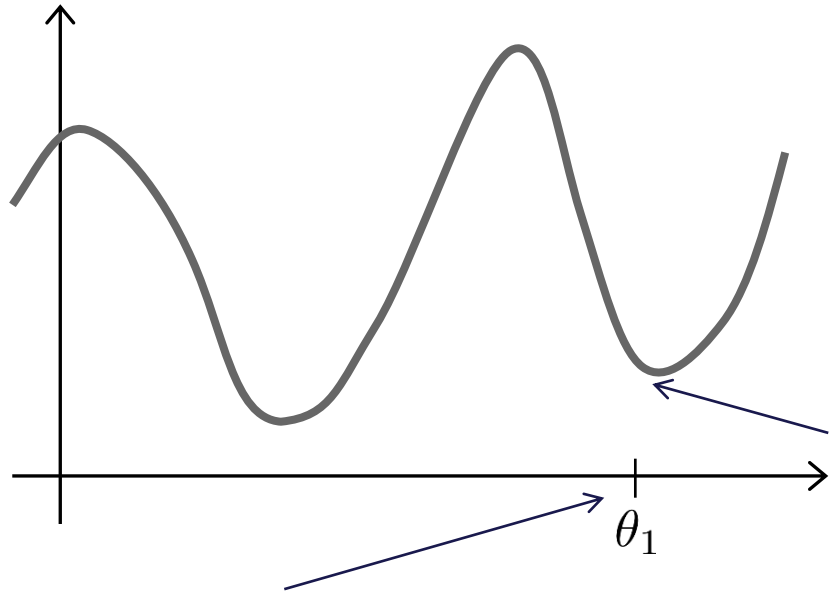
The limiting case is when x_2 approaches x_1

$= \frac{\Delta y}{\Delta x} as \Delta x \rightarrow 0 = \frac{dy}{dx}$

- In addition to α, the steepness of the gradient also control the step size.
- The step sizes become smaller as we get closer to the minimum, even with a fixed α.
- No need to decrease α with time or number of steps



The problem of local optima



$$\theta_1 := \theta_1 - \alpha \frac{d}{d\theta_1} J(\theta_1)$$

Gradient descent algorithm

repeat until convergence {

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta_0, \theta_1) \qquad (\text{for } j = 0 \text{ and } j = 1)$$

}

(Where α is the learning rate. E.g., 1, 0.1, 0.01, 0.001,… etc.)

Correct: **Simultaneous update**

$$\text{temp0} := \theta_0 - \alpha \frac{\partial}{\partial \theta_0} J(\theta_0, \theta_1)$$
$$\text{temp1} := \theta_1 - \alpha \frac{\partial}{\partial \theta_1} J(\theta_0, \theta_1)$$
$$\theta_0 := \text{temp0}$$
$$\theta_1 := \text{temp1}$$

Incorrect:

$$\text{temp0} := \theta_0 - \alpha \frac{\partial}{\partial \theta_0} J(\theta_0, \theta_1)$$
$$\theta_0 := \text{temp0}$$
$$\text{temp1} := \theta_1 - \alpha \frac{\partial}{\partial \theta_1} J(\theta_0, \theta_1)$$
$$\theta_1 := \text{temp1}$$

Gradient descent algorithm

repeat until convergence {

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta_0, \theta_1) \qquad \text{(simultaneously update } j = 0 \text{ and } j = 1\text{)}$$

}

Gradient descent algorithm

repeat until convergence {

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta_0, \theta_1)$$

(for $j = 1$ and $j = 0$)

}

Linear Regression Model

$$h_\theta(x) = \theta_0 + \theta_1 x$$

$$J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^{m} \left(h_\theta(x^{(i)}) - y^{(i)} \right)^2$$

$$\frac{\partial}{\partial \theta_j} J(\theta_0, \theta_1) = \frac{\partial}{\partial \theta_j} \frac{1}{2m} \sum_{i=1}^{m} (h_\theta(x^{(i)}) - y^{(i)})^2$$

$$\frac{\partial}{\partial \theta_j} J(\theta_0, \theta_1) = \frac{\partial}{\partial \theta_j} \frac{1}{2m} \sum_{i=1}^{m} \left(\theta_0 + \theta_1 x^{(i)} - y^{(i)}\right)^2$$

$$j = 0 : \frac{\partial}{\partial \theta_0} J(\theta_0, \theta_1) = \frac{1}{m} \sum_{i=1}^{m} \left(h_\theta(x^{(i)}) - y^{(i)}\right)$$

$$j = 0 : \frac{\partial}{\partial \theta_0} J(\theta_0, \theta_1) = \frac{1}{m} \sum_{i=1}^{m} \left(\theta_0 + \theta_1 x^{(i)} - y^{(i)}\right)$$

$$j = 1 : \frac{\partial}{\partial \theta_1} J(\theta_0, \theta_1) = \frac{1}{m} \sum_{i=1}^{m} \left(h_\theta(x^{(i)}) - y^{(i)}\right) x^{(i)}$$

$$j = 1 : \frac{\partial}{\partial \theta_1} J(\theta_0, \theta_1) = \frac{1}{m} \sum_{i=1}^{m} \left(\theta_0 + \theta_1 x^{(i)} - y^{(i)}\right) x^{(i)}$$

Gradient descent algorithm

repeat until convergence {

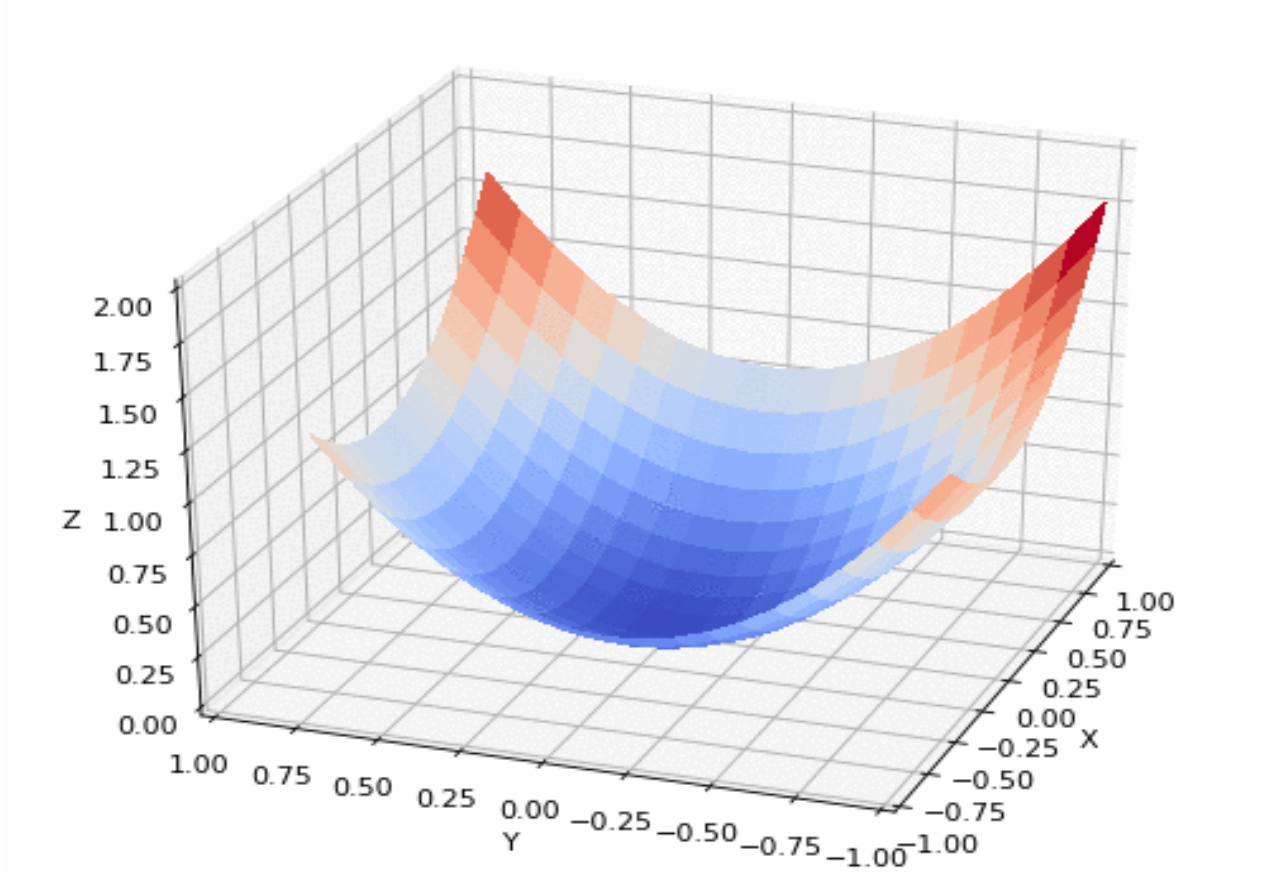
$$\theta_0 := \theta_0 - \alpha \frac{1}{m} \sum_{i=1}^{m} \left(h_\theta(x^{(i)}) - y^{(i)} \right)$$

$$\theta_1 := \theta_1 - \alpha \frac{1}{m} \sum_{i=1}^{m} \left(h_\theta(x^{(i)}) - y^{(i)} \right) \cdot x^{(i)}$$

}

update θ_0 and θ_1 simultaneously

Convex function



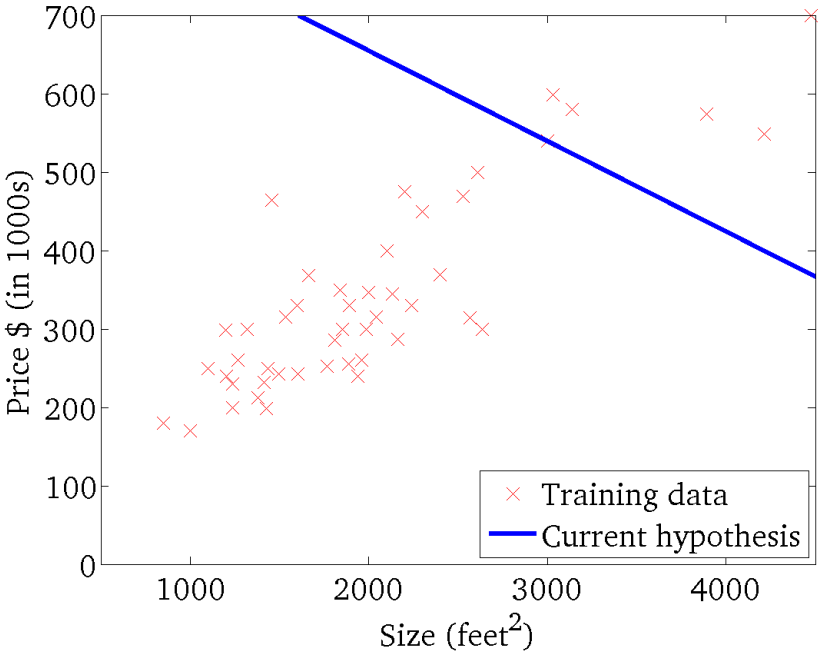
https://github.com/pvigier/gradient-descent

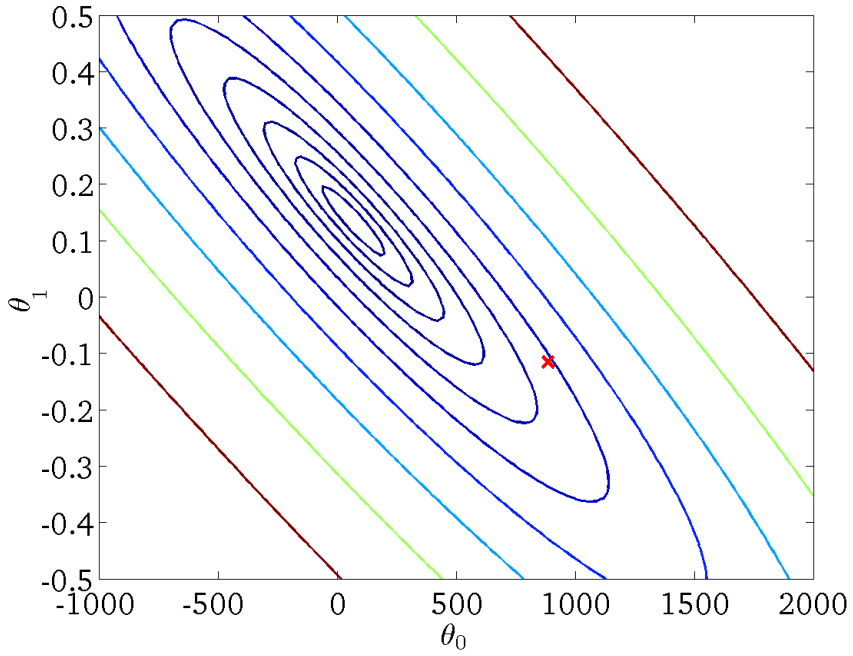
$$h_\theta(x) = \theta_0 + \theta_1 x$$

(for fixed θ_0, θ_1, this is a function of x)

$$J(\theta_0, \theta_1)$$

(function of the parameters θ_0, θ_1)



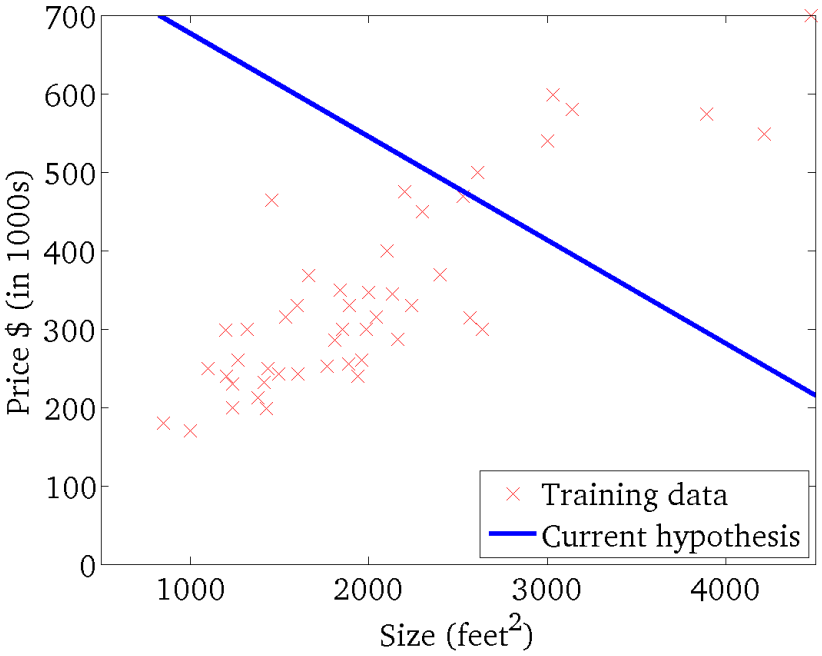


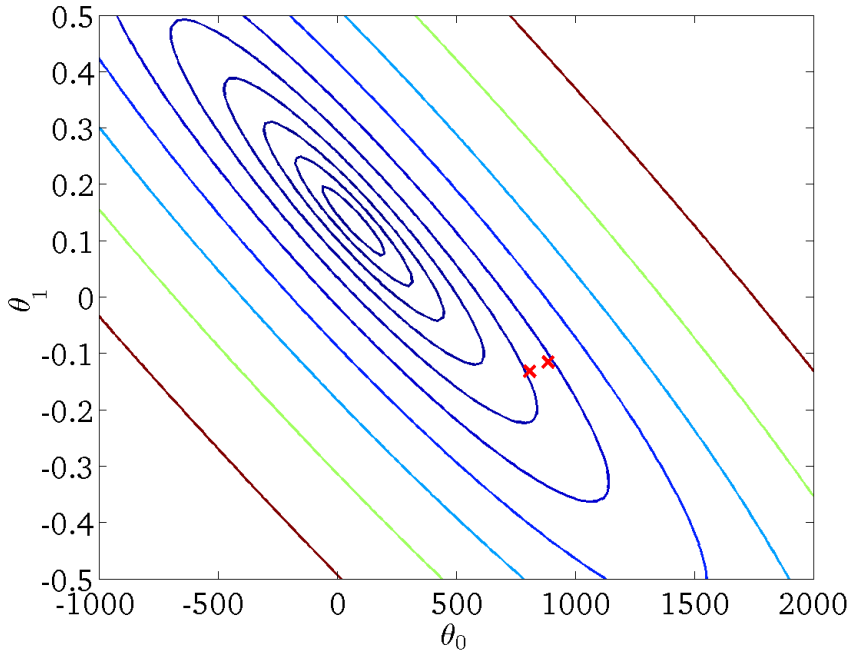
$$h_\theta(x) = \theta_0 + \theta_1 x$$

(for fixed θ_0, θ_1, this is a function of x)

$$J(\theta_0, \theta_1)$$

(function of the parameters θ_0, θ_1)



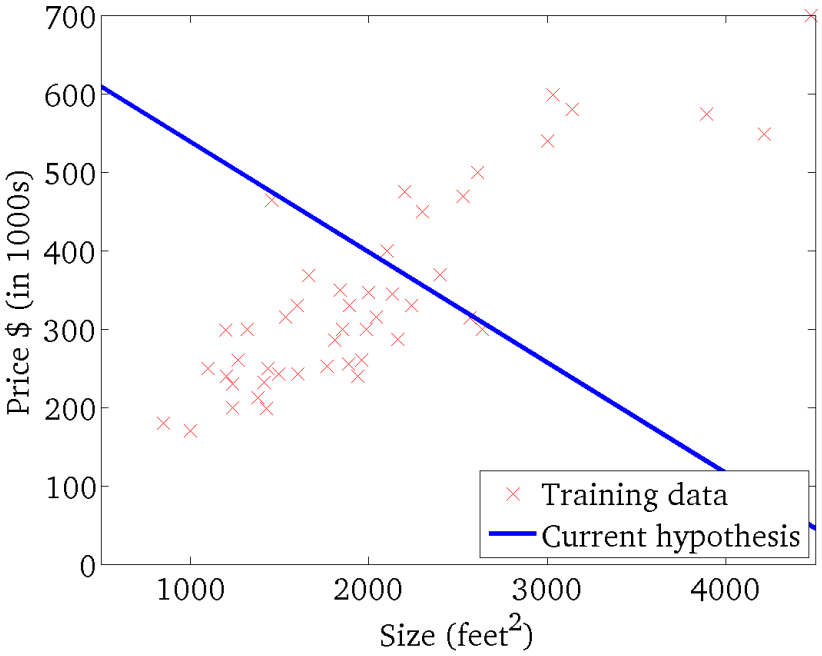


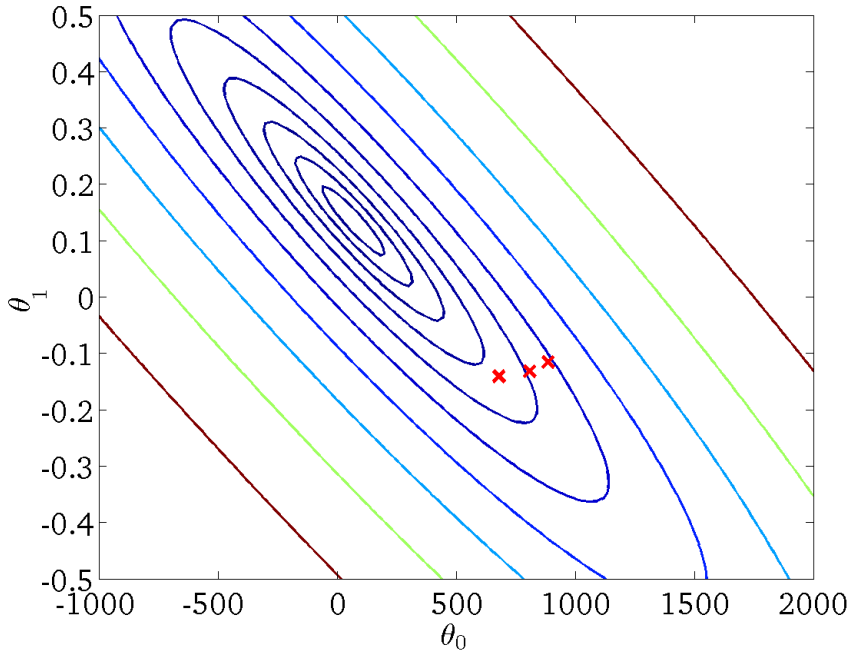
$$h_\theta(x) = \theta_0 + \theta_1 x$$

(for fixed θ_0, θ_1, this is a function of x)

$$J(\theta_0, \theta_1)$$

(function of the parameters θ_0, θ_1)



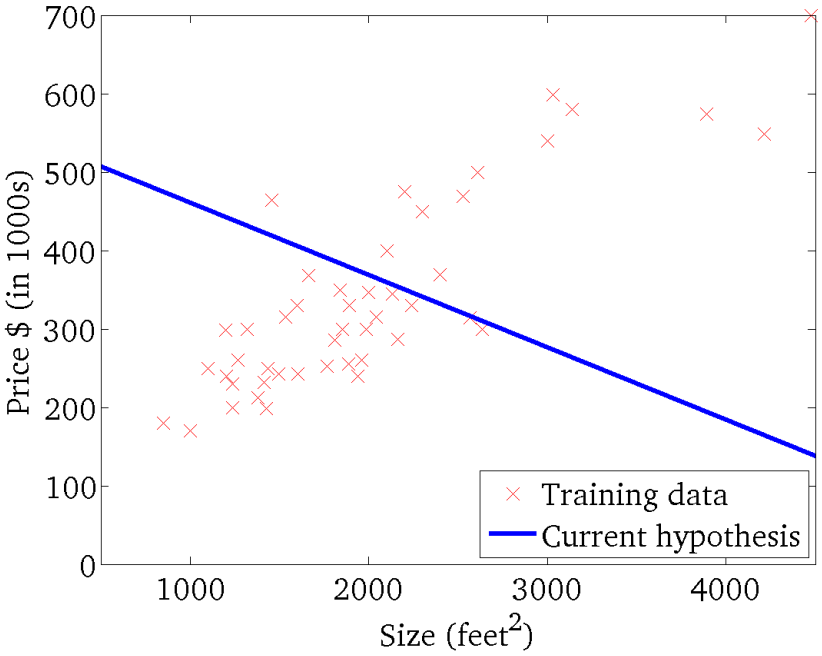


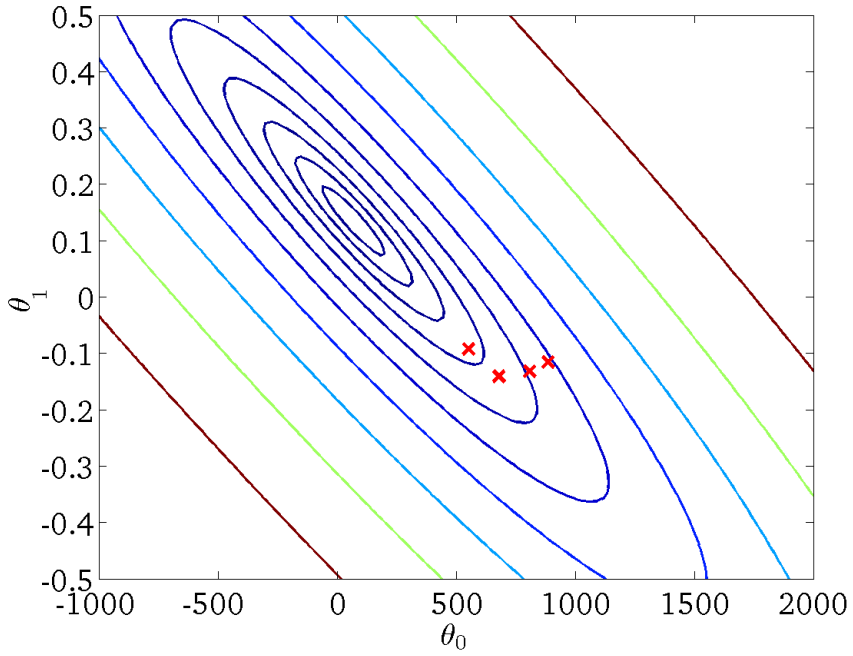
$$h_\theta(x) = \theta_0 + \theta_1 x$$

(for fixed θ_0, θ_1, this is a function of x)

$$J(\theta_0, \theta_1)$$

(function of the parameters θ_0, θ_1)



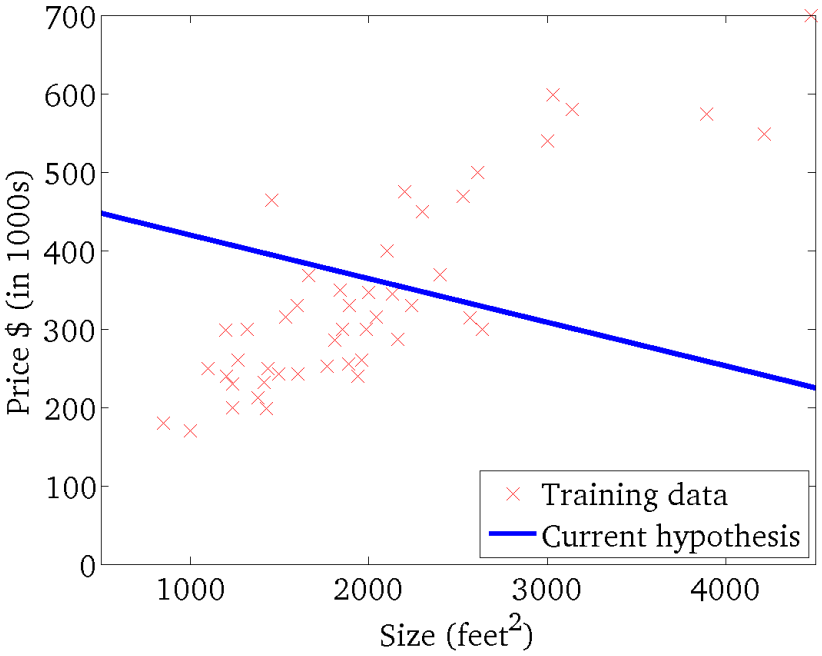


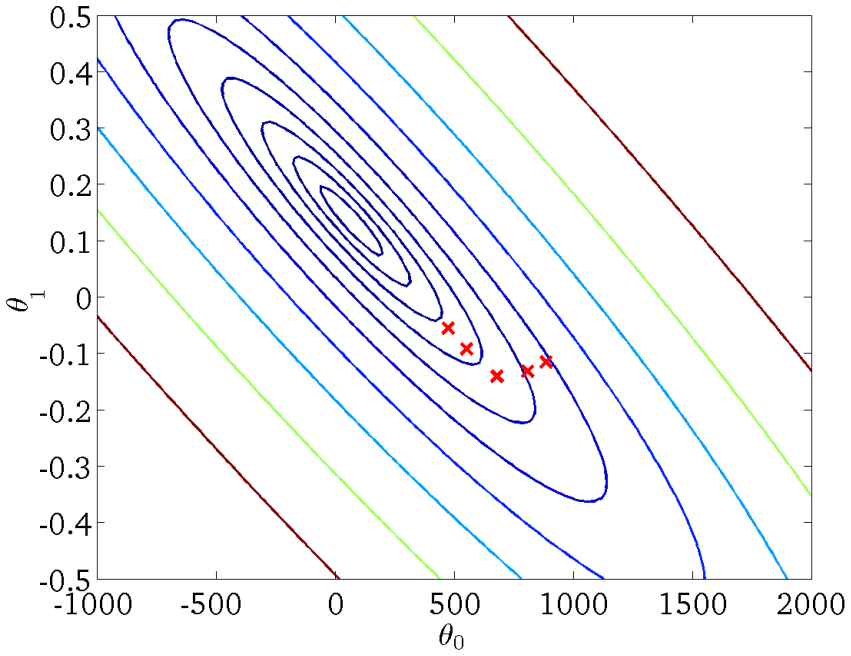
$$h_\theta(x) = \theta_0 + \theta_1 x$$

(for fixed θ_0, θ_1, this is a function of x)

$$J(\theta_0, \theta_1)$$

(function of the parameters θ_0, θ_1)



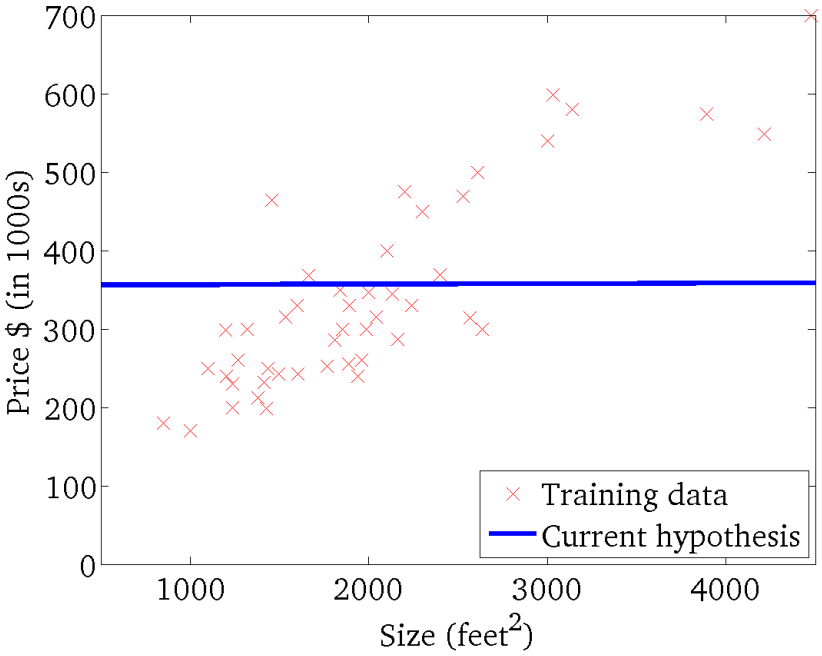


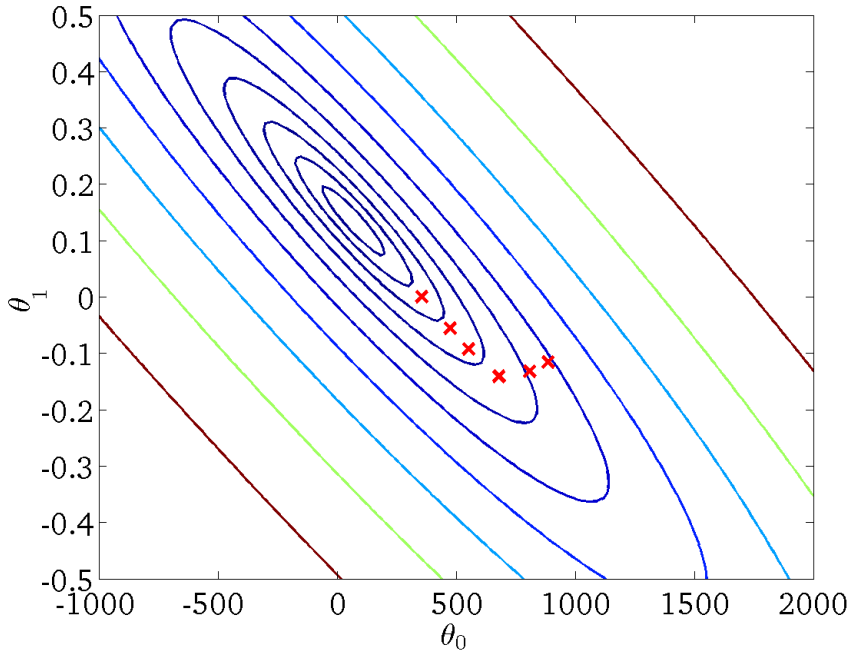
$$h_\theta(x) = \theta_0 + \theta_1 x$$

(for fixed θ_0, θ_1, this is a function of x)

$$J(\theta_0, \theta_1)$$

(function of the parameters θ_0, θ_1)



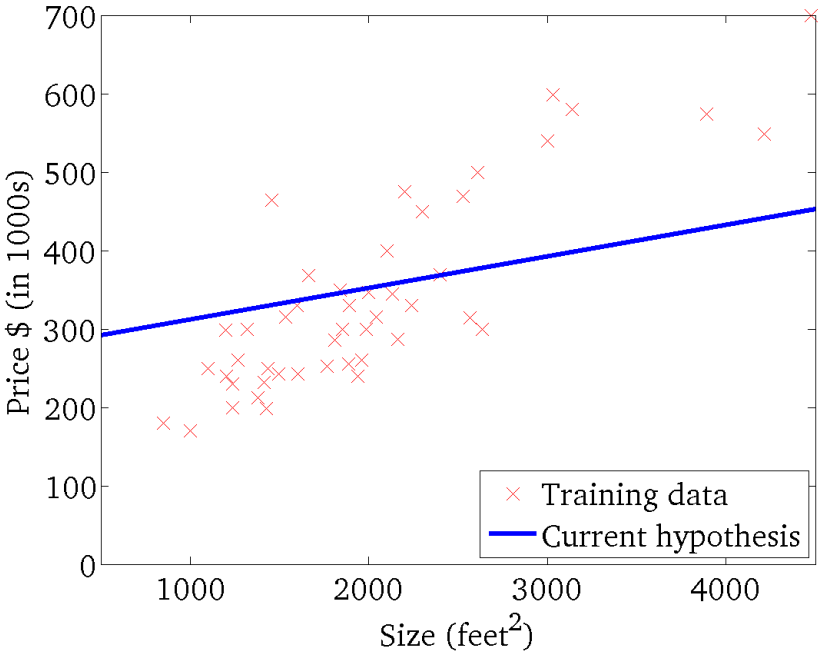


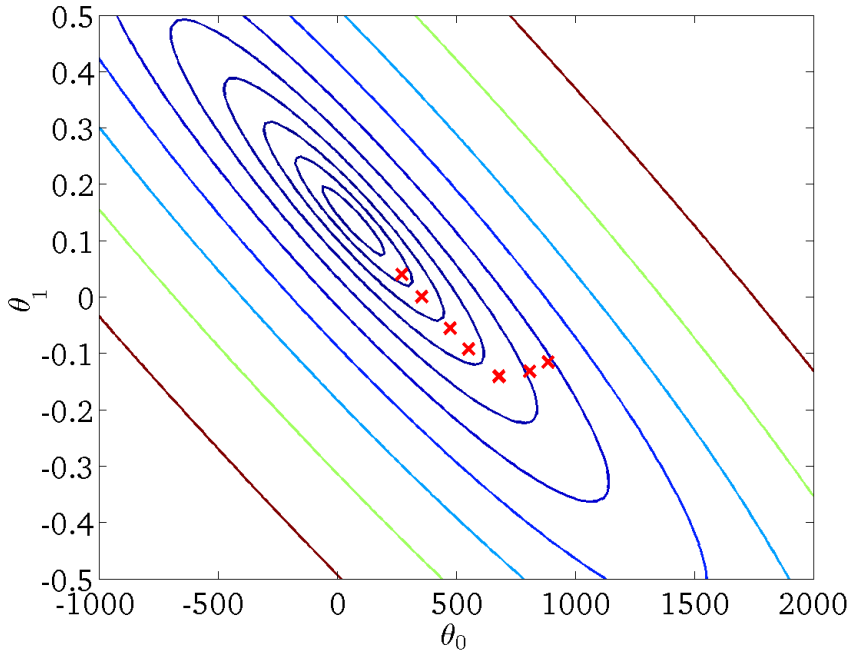
$$h_\theta(x) = \theta_0 + \theta_1 x$$

(for fixed θ_0, θ_1, this is a function of x)

$$J(\theta_0, \theta_1)$$

(function of the parameters θ_0, θ_1)



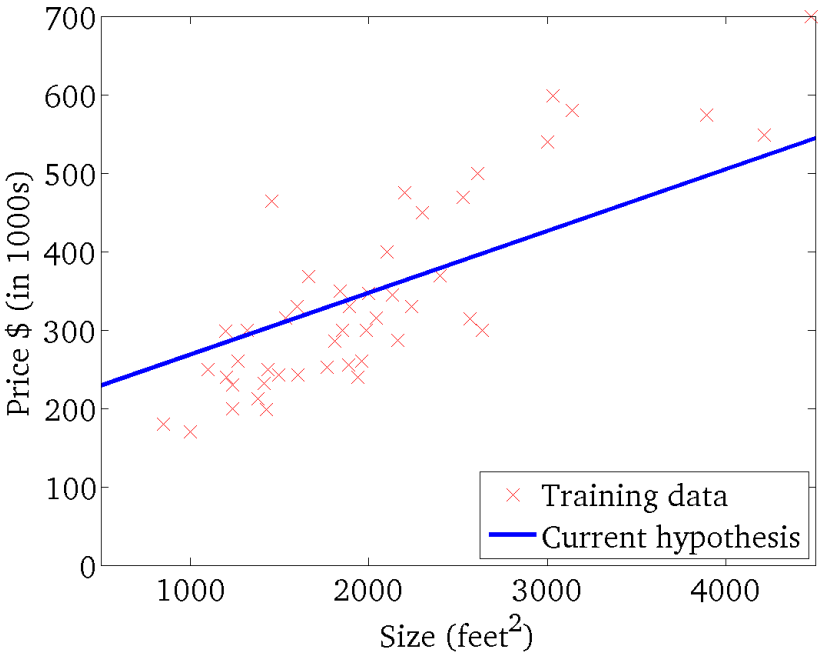


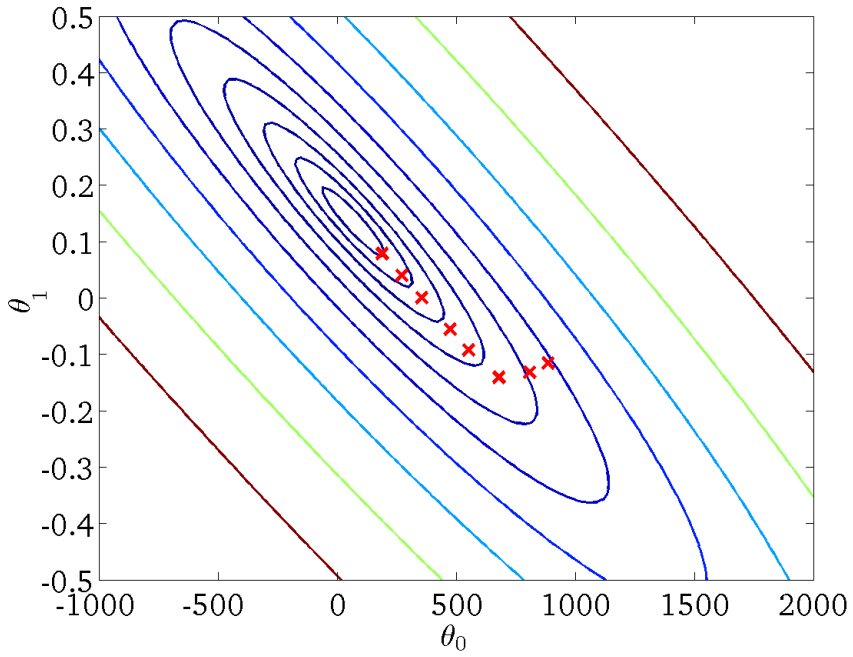
$$h_\theta(x) = \theta_0 + \theta_1 x$$

(for fixed θ_0, θ_1, this is a function of x)

$$J(\theta_0, \theta_1)$$

(function of the parameters θ_0, θ_1)



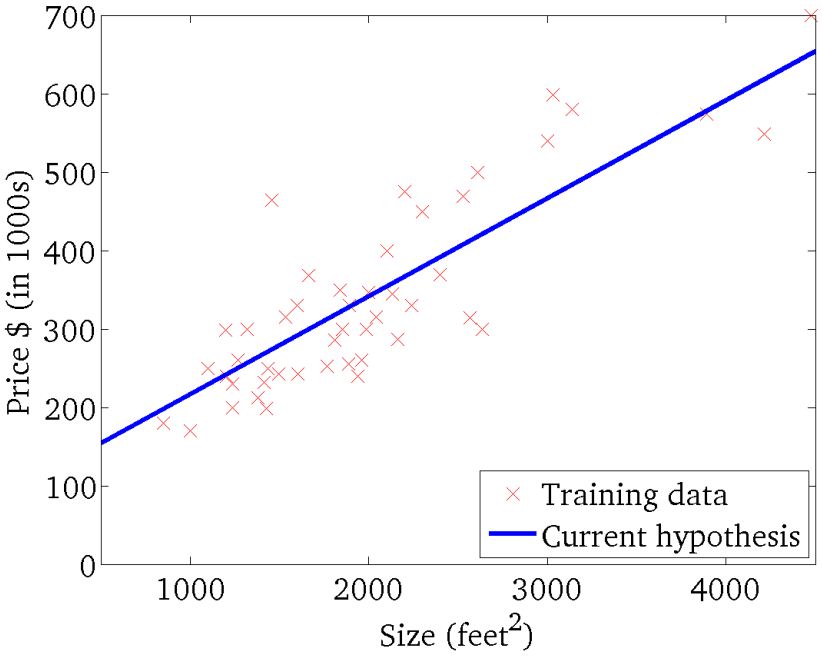


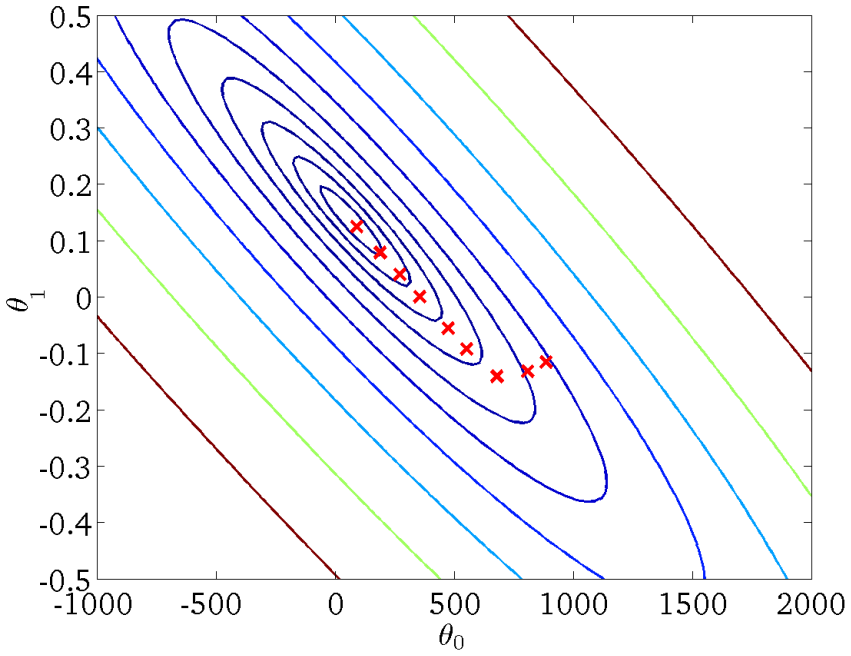
$$h_\theta(x) = \theta_0 + \theta_1 x$$

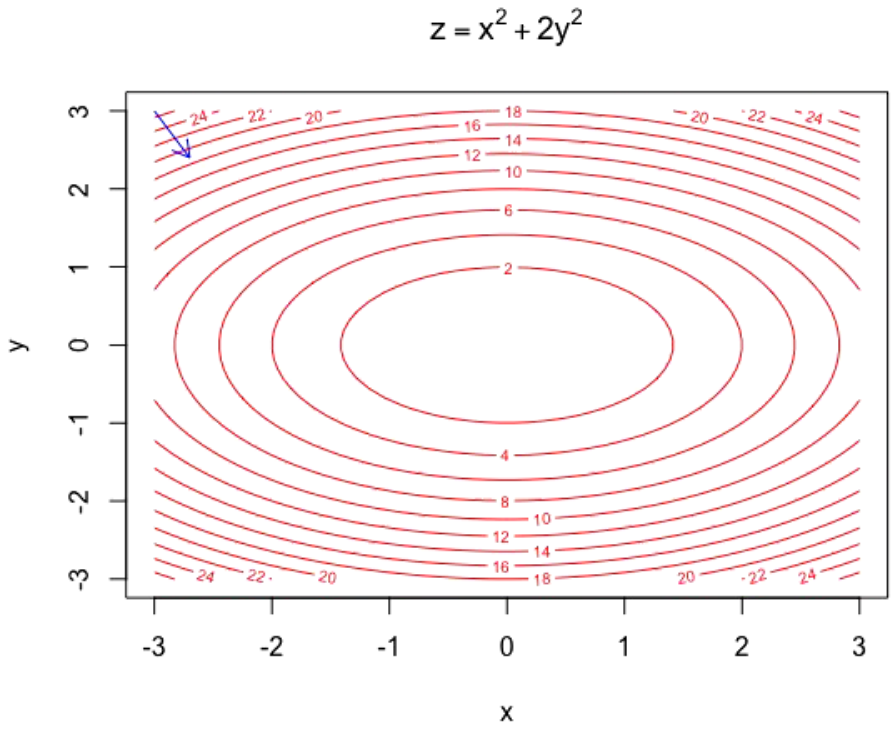
(for fixed θ_0, θ_1, this is a function of x)

$$J(\theta_0, \theta_1)$$

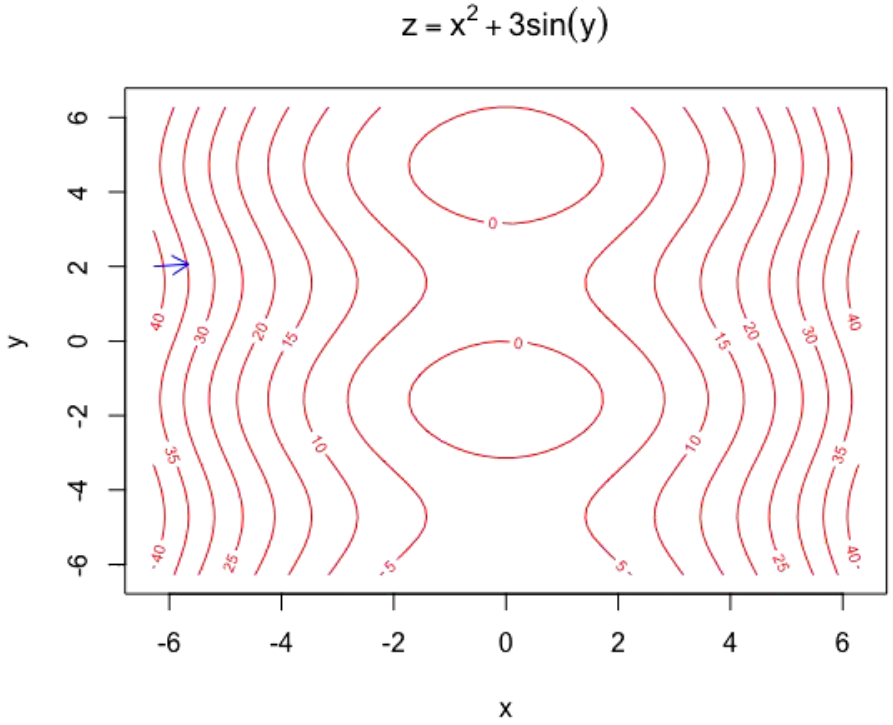
(function of the parameters θ_0, θ_1)



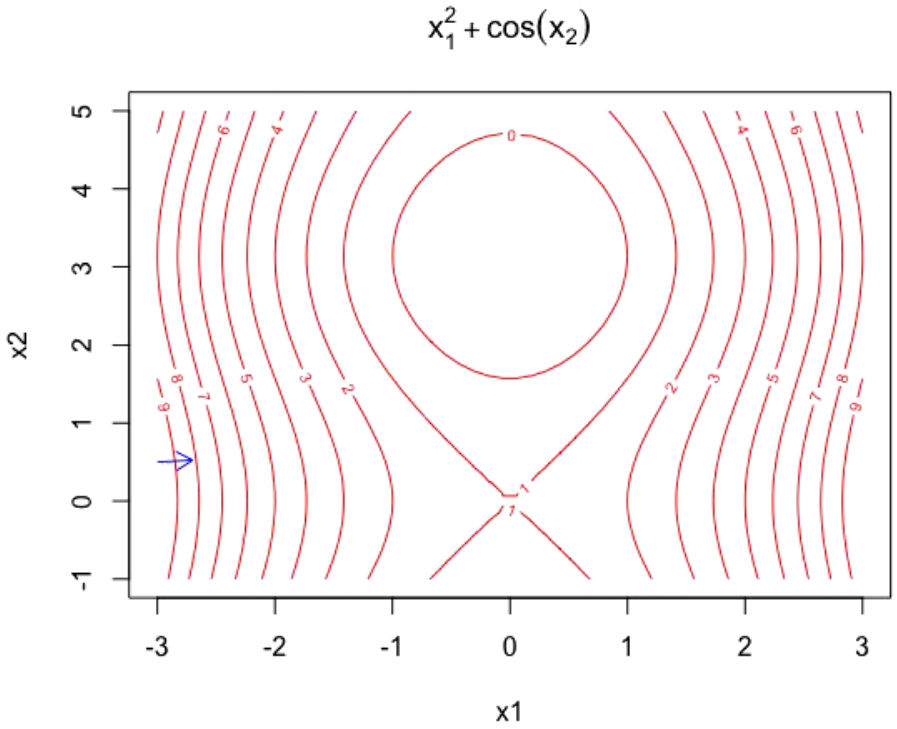




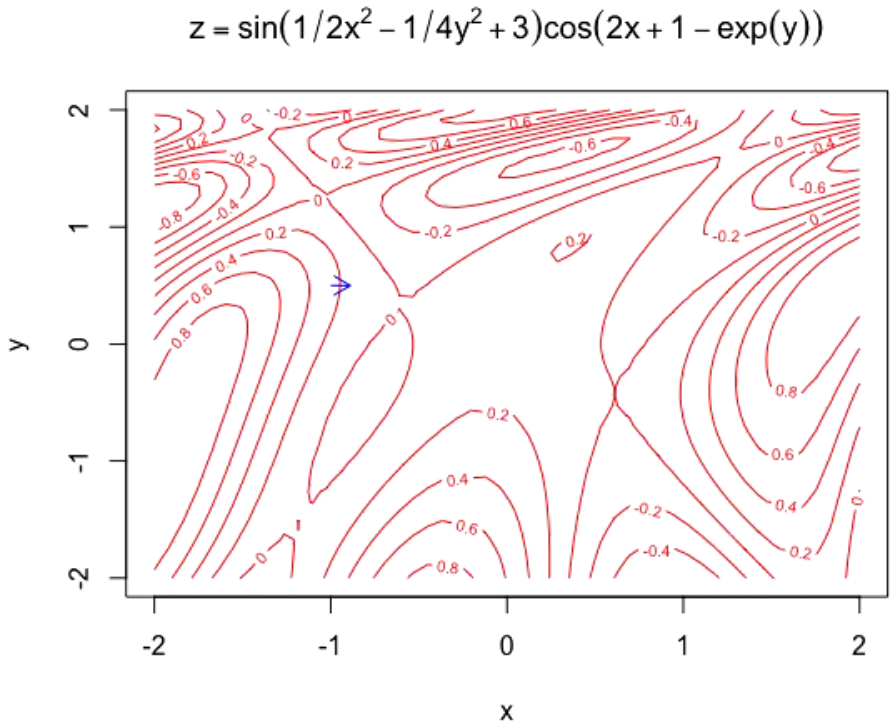
https://yihui.org/animation/example/grad-desc/



https://yihui.org/animation/example/grad-desc/



https://yihui.org/animation/example/grad-desc/



https://yihui.org/animation/example/grad-desc/

“Batch” Gradient Descent

“Batch”: Each step of gradient descent uses all the training examples.

$$
\begin{aligned}
&\text{repeat until convergence } \{ \\
&\quad \theta_0 := \theta_0 - \alpha \frac{1}{m} \sum_{i=1}^{m} \left(h_\theta(x^{(i)}) - y^{(i)} \right) \\
&\quad \theta_1 := \theta_1 - \alpha \frac{1}{m} \sum_{i=1}^{m} \left(h_\theta(x^{(i)}) - y^{(i)} \right) \cdot x^{(i)} \\
&\}
\end{aligned}
$$

Types of Gradient Descent

Batch gradient descent: Computes the gradient of the cost function with respect to to the parameters θ for the entire training dataset (m instances).

Stochastic gradient descent: SGD performs a parameter update for *each* training example $x^{(i)}$ and label $y^{(i)}$

Mini-batch gradient descent: Mini-batch gradient descent takes the best of both worlds and performs an update for random mini-batches of k training examples, where $k < m$.

Read: https://ruder.io/optimizing-gradient-descent/

LUMS

CENTER FOR SPEECH AND LANGUAGE TECHNOLOGIES

Linear Regression
Multivariate

Agha Ali Raza

CS535/EE514 – Machine Learning

References

- Machine Learning, Andrew Ng, on Coursera by Stanford – a https://www.coursera.org/learn/machine-learning
- Deep Learning Specialization, Andrew Ng, on Coursera by deeplearning.ai – https://www.coursera.org/specializations/deep-learning

Single feature (variable)

Size (feet^2)	Price ($1000)
x	y
2104	460
1416	232
1534	315
852	178
…	…

$$h_\theta(x) = \theta_0 + \theta_1 x$$

Andrew Ng

Multiple features (variables)

Size (feet²)	Number of bedrooms	Number of floors	Age of home (years)	Price ($1000)
2104	5	1	45	460
1416	3	2	40	232
1534	3	2	30	315
852	2	1	36	178
…	…	…	…	…

$$h_\theta(X) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_3 + \cdots + \theta_n x_n$$

Notation:

- **m** = Number of training samples
- **n** = Number of features
- **x** = Feature
- **y** = Label
- **$(x_j^{(i)}, y_j^{(i)})$:** the jth feature of the ith sample in the dataset

Andrew Ng

Hypothesis

Previously:

$$\boldsymbol{h_\theta(x) = \theta_0 + \theta_1 x}$$

Now:

$$\boldsymbol{h_\theta(X) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_3 + \cdots + \theta_n x_n}$$

where, $X = [x_1\ x_2\ \ldots\ x_n]$ is the n-dimensional feature vector, and $\Theta = [\theta_0\ \theta_1\ \ldots\ \theta_n]$ is an (n+1)-dimensional vector of weights.

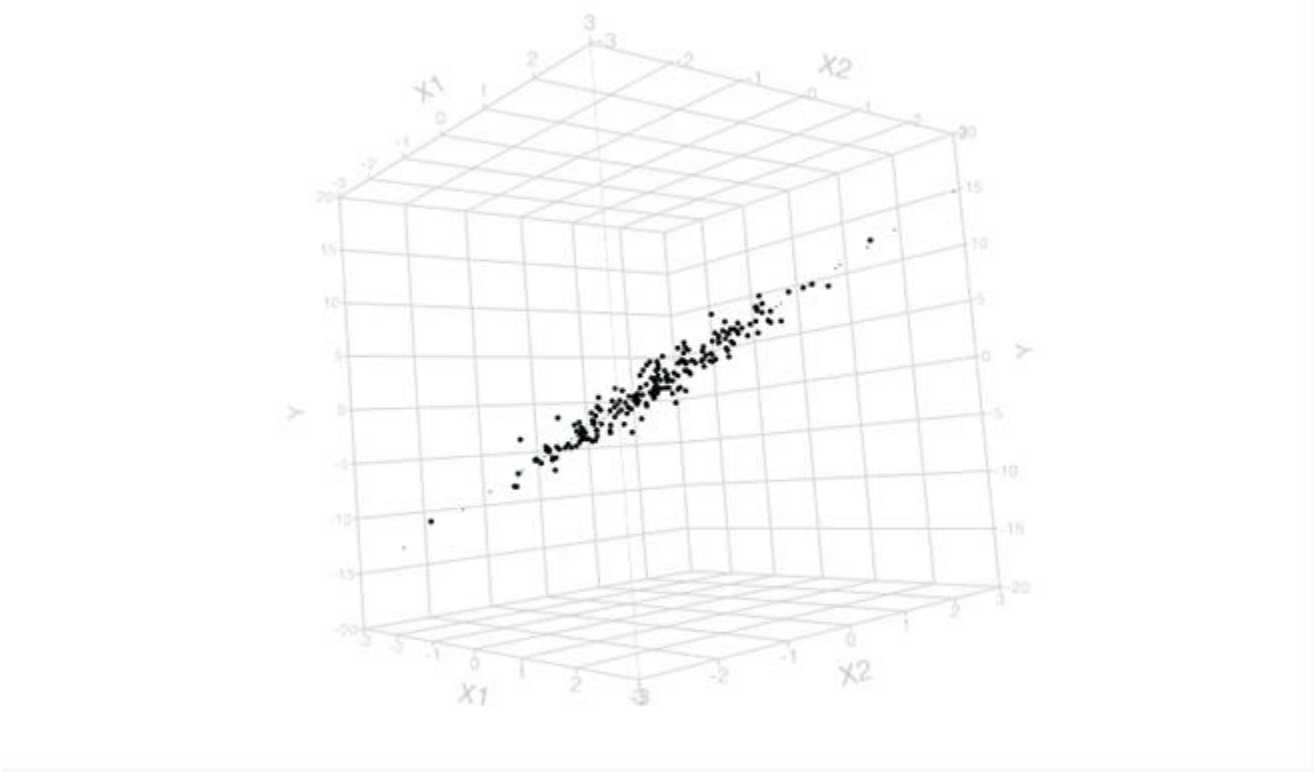
$$\boldsymbol{X \in \mathbb{R}^n, \Theta \in \mathbb{R}^{n+1}}$$

Geometric Interpretation

$$h_\theta(X) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_3 + \cdots + \theta_n x_n$$

where, $X = [x_1\ x_2\ \ldots\ x_n]$ is the n-dimensional feature vector, and $\Theta = [\theta_0\ \theta_1\ \ldots\ \theta_n]$ is an (n+1)-dimensional vector of weights.

$$h_\theta(X) = \theta_0 + \theta_1 x_1 + \theta_2 x_2$$ (a 2-D hyperplane in 3-D)



https://www.jmp.com/en_us/statistics-knowledge-portal/what-is-multiple-regression/fitting-multiple-regression-model.html

Hypothesis

$$\boldsymbol{h_\theta(X) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_3 + \cdots + \theta_n x_n}$$

where, $X = [x_1\ x_2\ \ldots\ x_n]$ is the n-dimensional feature vector, and $\Theta = [\theta_0\ \theta_1\ \ldots\ \theta_n]$ is an (n+1)-dimensional vector of weights.

$$\boldsymbol{X \in \mathbb{R}^n, \Theta \in \mathbb{R}^{n+1}}$$

To make this more uniform, assume $x_0 = 1$ to get:

$$\boldsymbol{h_\theta(x) = \theta_0 x_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_3 + \cdots + \theta_n x_n}$$

where, $X = [x_0\ x_1\ x_2\ \ldots\ x_n]$ is the (n+1)-dimensional feature vector, and $\Theta = [\theta_0\ \theta_1\ \ldots\ \theta_n]$ is an (n+1)-dimensional vector of weights.

$$\boldsymbol{X \in \mathbb{R}^{n+1}, \Theta \in \mathbb{R}^{n+1}}$$

Vectorizing the notation

$$\boldsymbol{h_\theta(x) = \theta_0 x_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_3 + \cdots + \theta_n x_n}$$

Now we can redefine our hypothesis as:

$\Theta = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \vdots \\ \theta_n \end{bmatrix}, X = \begin{bmatrix} x_0 \\ x_1 \\ \vdots \\ x_n \end{bmatrix}$ and where $\Theta \in \mathbb{R}^{n+1}$ and $X \in \mathbb{R}^{n+1}$

and,

$$\boldsymbol{h_\Theta(X) = \Theta^T X}$$

Multivariate linear regression

For m-training instances

$$h_\theta(x) = \theta_0 x_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_3 + \cdots + \theta_n x_n$$

$$\Theta = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \vdots \\ \theta_n \end{bmatrix}, X = \begin{bmatrix} x_0^{(1)} & x_0^{(2)} & x_0^{(3)} & & x_0^{(m)} \\ x_1^{(1)} & x_1^{(2)} & x_1^{(3)} & \dots & x_1^{(m)} \\ \dots & \dots & \dots & & \dots \\ x_n^{(1)} & x_n^{(2)} & x_n^{(3)} & & x_n^{(m)} \end{bmatrix}$$

$n \times 1$ $\qquad$ $n \times m$

$$h_\Theta(X) = \Theta^T X = [\theta_0\ \theta_1\ \dots \theta_n] \begin{bmatrix} x_0^{(1)} & x_0^{(2)} & x_0^{(3)} & & x_0^{(m)} \\ x_1^{(1)} & x_1^{(2)} & x_1^{(3)} & \dots & x_1^{(m)} \\ \dots & \dots & \dots & & \dots \\ x_n^{(1)} & x_n^{(2)} & x_n^{(3)} & & x_n^{(m)} \end{bmatrix} = [h_\theta(x^{(0)})\ h_\theta(x^{(1)})\ \dots\ h_\theta(x^{(m)})]$$

$1 \times n$ $\qquad$ $n \times m$ $\qquad$ $=$ $\qquad$ $1 \times m$

LUMS

CENTER FOR SPEECH AND LANGUAGE TECHNOLOGIES

Linear Regression
Multivariate Gradient Descent

Agha Ali Raza

CS535/EE514 – Machine Learning

Hypothesis: $h_\theta(x) = h_\Theta(X) = \Theta^T X = \theta_0 x_0 + \theta_1 x_1 + \cdots + \theta_n x_n$

Parameter vector: $\Theta = \theta_0, \theta_1 \ldots \theta_n$

Feature vector: $x = x_0 x_1 \cdots x_n$

Cost Function: $J(\Theta) = \frac{1}{2m} \sum_{i=1}^{m} \left(h_\theta(x^{(i)}) - y^{(i)} \right)^2$

Gradient descent:

repeat{

$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} \left(J(\Theta) \right)$ //simultaneously update for all j = 0 … n

}

Gradient Descent

Previously (n=1):

Repeat $\{$

$$\theta_0 := \theta_0 - \alpha \underbrace{\frac{1}{m}\sum_{i=1}^{m}(h_\theta(x^{(i)}) - y^{(i)})}_{\frac{\partial}{\partial\theta_0}J(\theta)}$$

$$\theta_1 := \theta_1 - \alpha \frac{1}{m}\sum_{i=1}^{m}(h_\theta(x^{(i)}) - y^{(i)})x^{(i)}$$

(simultaneously update θ_0, θ_1)

$\}$

New algorithm $(n \geq 1)$:

Repeat $\{$

$$\theta_j := \theta_j - \alpha \frac{1}{m}\sum_{i=1}^{m}(h_\theta(x^{(i)}) - y^{(i)})x_j^{(i)}$$

(simultaneously update θ_j for $j = 0, \ldots, n$)

$\}$

$$\theta_0 := \theta_0 - \alpha \frac{1}{m}\sum_{i=1}^{m}(h_\theta(x^{(i)}) - y^{(i)})x_0^{(i)}$$

$$\theta_1 := \theta_1 - \alpha \frac{1}{m}\sum_{i=1}^{m}(h_\theta(x^{(i)}) - y^{(i)})x_1^{(i)}$$

$$\theta_2 := \theta_2 - \alpha \frac{1}{m}\sum_{i=1}^{m}(h_\theta(x^{(i)}) - y^{(i)})x_2^{(i)}$$

$\ldots$

Andrew Ng

LUMS

CENTER FOR SPEECH AND LANGUAGE TECHNOLOGIES

Linear Regression
Practical Issues

Agha Ali Raza

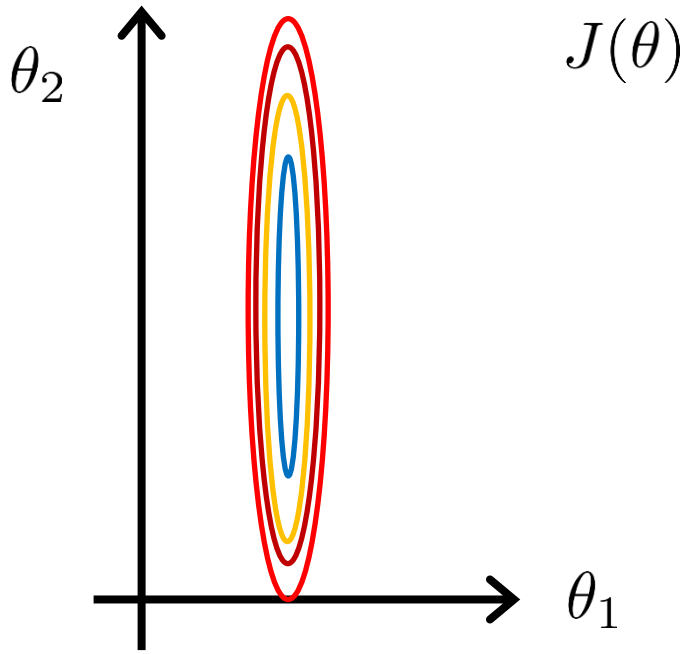
CS535/EE514 – Machine Learning

Feature Scaling

- Make sure features are on a similar scale or the learning will occur very slowly.
- As α needs to be small enough for the dimension with the smallest scale.

E.g. x_1 = size (0-2000 feet^2)

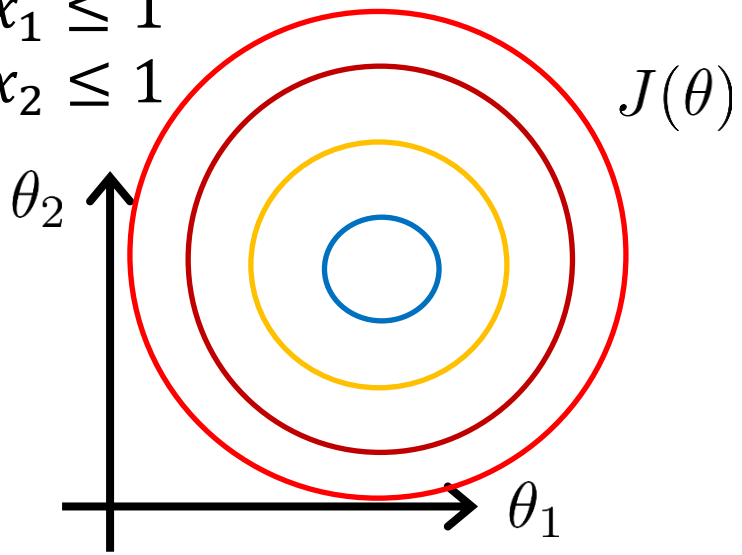
x_2 = number of bedrooms (1-5)



$$x_1 = \frac{\text{size (feet}^2\text{)}}{2000}$$

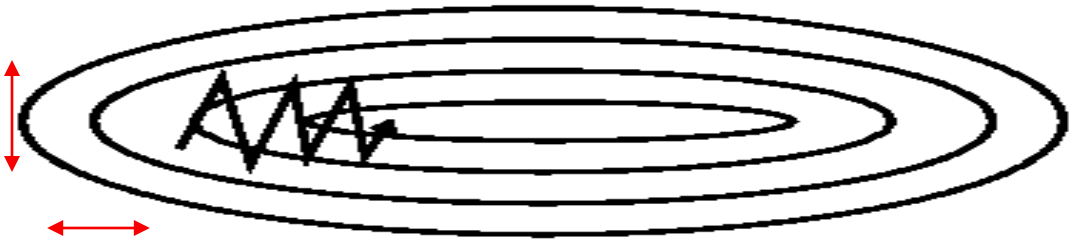
$$x_2 = \frac{\text{number of bedrooms}}{5}$$

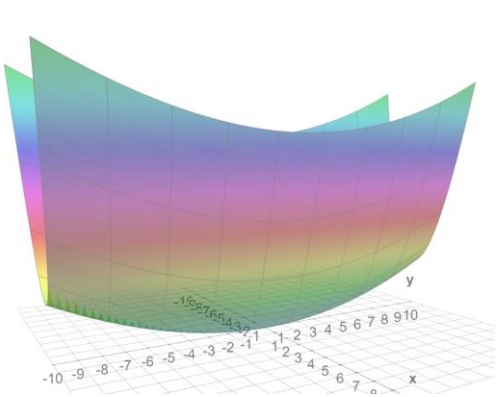
$$0 \le x_1 \le 1$$
$$0 \le x_2 \le 1$$

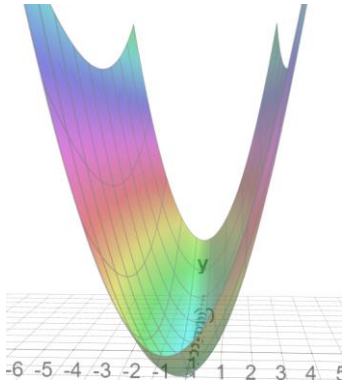


Ravines

Gradient descent has trouble navigating ravines, i.e., areas where the surface curves much more steeply in one dimension than in another, which are common around local optima.







A possible solution: Adding momentum

- Slow down the movement along the dimension where the gradients are rapidly changing direction
- Gather speed along the dimension where the gradients are lining up

Gradient descent with momentum

- Instead of the following step:

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta)$$

$$\theta_j := \theta_j - \alpha d\theta_j$$

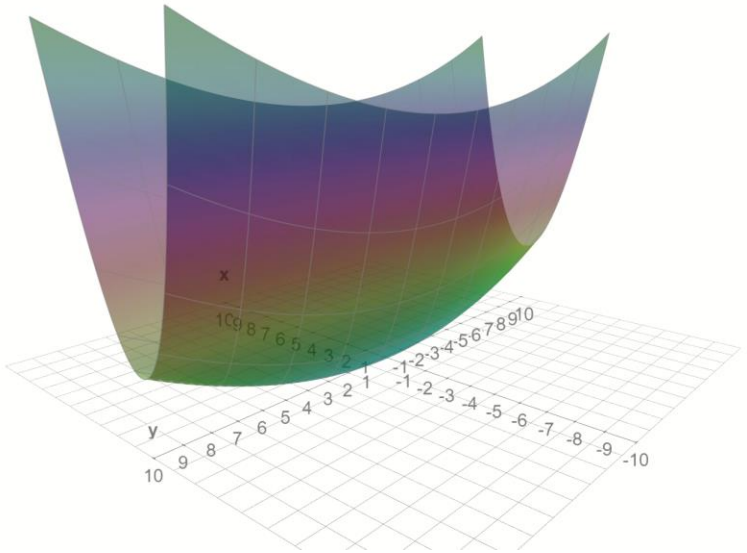
- Calculate:

$$v_{d\theta_j} := \beta v_{d\theta_j} + (1-\beta) d\theta_j$$

- And use it to update the weights:

$$\theta_j := \theta_j - \alpha v_{d\theta_j}$$

- Usually, $\beta = 0.9$ works well in practice



https://www.math3d.org/

Momentum

- $\beta = 0.1$
  - At $n = 3$; the gradient at $t = 3$ will contribute 100% of its value, the gradient at $t = 2$ will contribute 10% of its value, and gradient at t=1 will only contribute 1% of its value.
  - Here contribution from earlier gradients decreases rapidly.
- $\beta = 0.9$
  - At $n = 3$; the gradient at $t = 3$ will contribute 100% of its value, $t = 2$ will contribute 90% of its value, and gradient at $t = 1$ will contribute 81% of its value.
  - A higher β will accommodate more gradients from the past. Hence, generally, β is kept around 0.9 in most of the cases.
- **Case 1:** When all the past gradients have the same sign
  - The summation term will become large, and we will take large steps while updating the weights.
- **Case 2:** When some of the gradients have positive sign whereas others have negative
  - The summation term will become small and weight updates will be small.
- Gradient Descent with Momentum takes small steps in directions where the gradients oscillate and take large steps along the direction where the past gradients have the same direction(same sign).

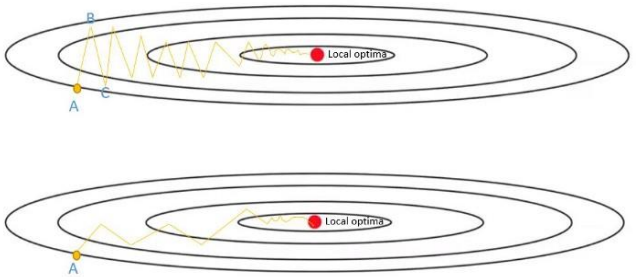
$$v(0) = 0$$

$$v(1) = \beta * v(0) + (1 - \beta) * \delta(1)$$
$$v(1) = (1 - \beta) * \delta(1)$$

$$v(2) = \beta * v(1) + (1 - \beta) * \delta(2)$$
$$v(2) = \beta * \{(1 - \beta) * \delta(1)\} + (1 - \beta) * \delta(2)$$
$$v(2) = (1 - \beta)\{\beta * \delta(1) + \delta(2)\}$$

$$v(3) = \beta * v(2) + (1 - \beta) * \delta(3)$$
$$v(3) = \beta * \{(1 - \beta)\{\beta * \delta(1) + \delta(2)\}\} + (1 - \beta) * \delta(3)$$
$$v(3) = (1 - \beta)\{\beta^2 * \delta(1) + \beta * \delta(2) + \delta(3)\}$$

$$v(n) = (1 - \beta) \sum_{t=1}^{n} \beta^{n-t} \delta(t)$$



https://towardsdatascience.com/gradient-descent-with-momentum-59420f626c8f#

Gradient Descent with Momentum

Initialize v_{dW} *and* v_{db} *to* 0

On iteration t:

Compute dW, db on the current mini-batch

$$v_{dW} = \beta v_{dW} + (1 - \beta) dW$$

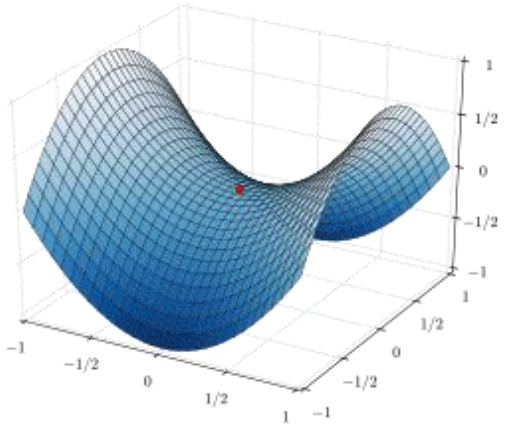
$$v_{db} = \beta v_{db} + (1 - \beta) db$$

$$W = W - \alpha v_{dW}, \quad b = b - \alpha v_{db}$$

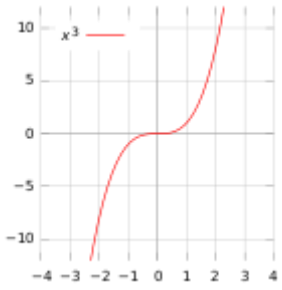
Hyperparameters: α, β $\beta = 0.9$

Gradient Descent With Momentum (C2W2L06), https://www.youtube.com/watch?v=k8fTYJPd3_I

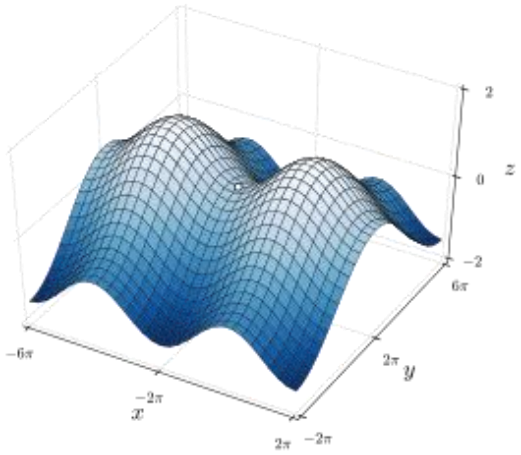
Saddle points



A saddle point (in red) on the graph of z=x2−y2 (hyperbolic paraboloid)



The plot of $y = x^3$ with a saddle point at 0



- A challenge of minimizing highly non-convex error functions is avoiding getting trapped in suboptimal local minima.
- The difficulty arises not only from local minima but also from saddle points, i.e., points where one dimension slopes up and other slopes down.
- These saddle points are usually surrounded by a plateau of the same error, which makes it notoriously hard for GD to escape, as the gradient is close to zero in all dimensions.
- Momentum will help here as well
  - Gradients from the past will push the cost further to move around a saddle point.

Mean normalization

Replace x_i with $x_i - \mu_i$ to make features have approximately zero mean (Do not apply to $x_0 = 1$).

E.g. $x_1 = \frac{size - 1000}{2000}$

$x_2 = \frac{\#bedrooms - 2}{5}$

$$-0.5 \leq x_1 \leq 0.5, -0.5 \leq x_2 \leq 0.5$$

Ideally: $x_1 = \frac{x_1 - \mu_1}{s_1}$, $x_2 = \frac{x_2 - \mu_2}{s_2}$

Andrew Ng

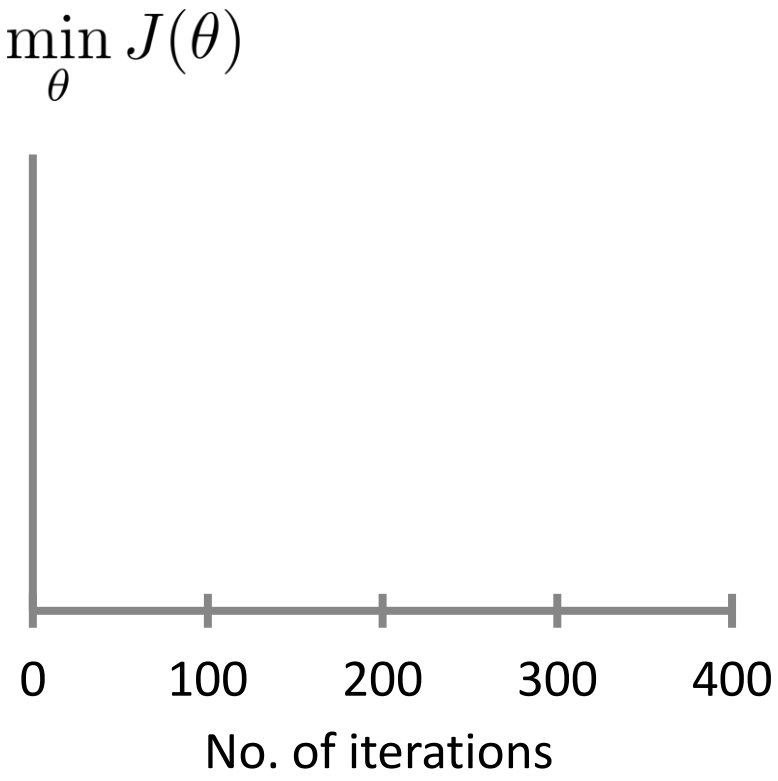
Gradient descent

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta)$$

- "Debugging": How to make sure gradient descent is working correctly.
- How to choose learning rate α.

Andrew Ng

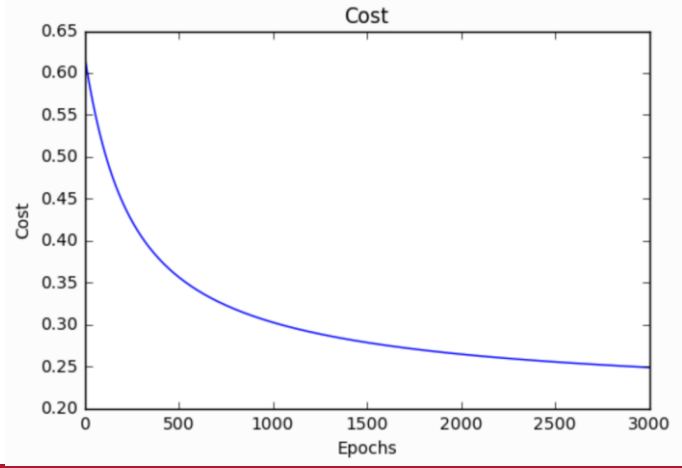
Making sure gradient descent is working correctly.



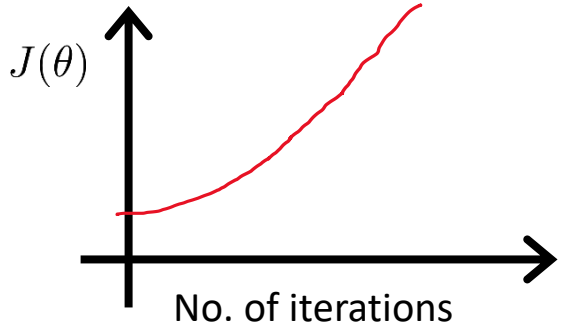
Ideally J(θ) should decrease after each iteration.

Example automatic convergence test:

Declare convergence if $J(\theta)$ decreases by less than 10^{-3} in one iteration.

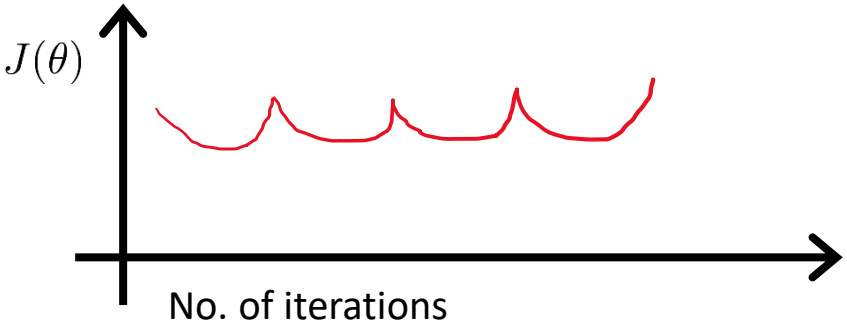


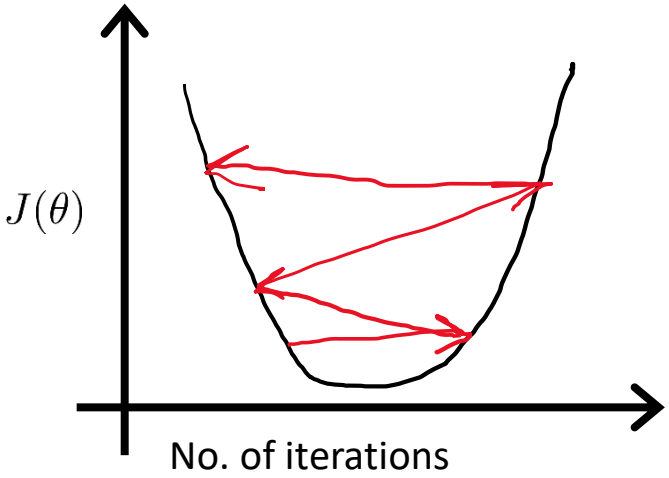
Making sure gradient descent is working correctly.



Gradient descent not working.

Use smaller α.





- For sufficiently small α, $J(\theta)$ should decrease on every iteration.
- But if α is too small, gradient descent can be slow to converge.

Andrew Ng

Summary:

- If α is too small: slow convergence.
- If α is too large: $J(\theta)$ may not decrease on every iteration; may not converge.

To choose α, try

$$\ldots, 0.001, \qquad , 0.01, \qquad , 0.1, \qquad , 1, \ldots$$

Andrew Ng

LUMS

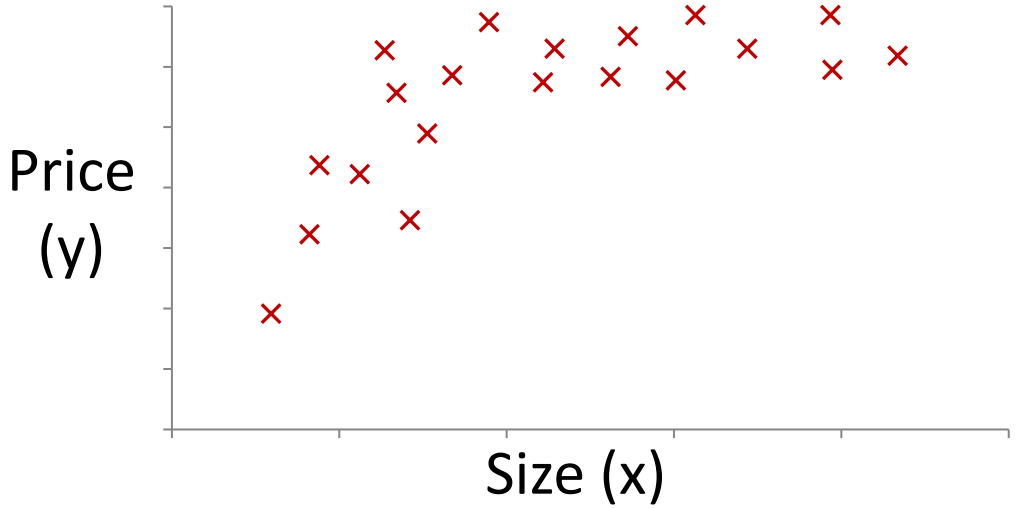
CENTER FOR SPEECH AND LANGUAGE TECHNOLOGIES

Linear Regression
Polynomial Regression

Agha Ali Raza

CS535/EE514 – Machine Learning

Polynomial regression



$$\theta_0 + \theta_1 x + \theta_2 x^2$$

$$\theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3$$

$$h_\theta(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_3$$
$$= \theta_0 + \theta_1 (size) + \theta_2 (size)^2 + \theta_3 (size)^3$$

$$x_1 = (size)$$
$$x_2 = (size)^2$$
$$x_3 = (size)^3$$

Feature scaling becomes important as now:

Size = 1 – 1000

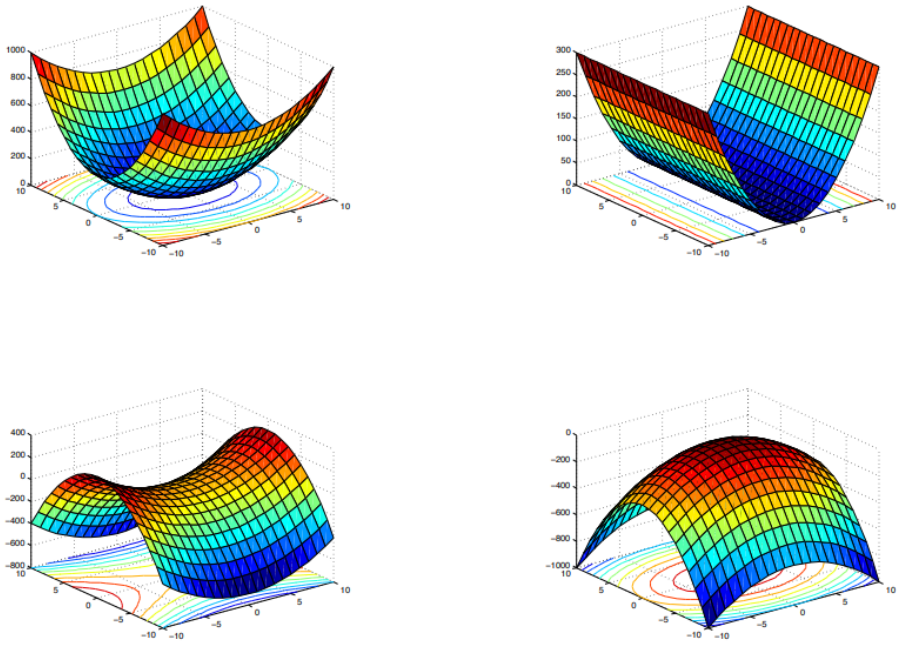
Size^2= 1 – 1000,000

Size3 = 1 – 1000,000,000

Andrew Ng

Example: Polynomial regression models with two predictor variables and interaction terms are quadratic forms. Their surfaces can have many different shapes depending on the values of the model parameters with the contour lines being either parallel lines, parabolas or ellipses.

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_{11} x_1^2 + \beta_{22} x_2^2 + \beta_{12} x_1 x_2 + \epsilon$$



http://mezeylab.cb.bscb.cornell.edu/labmembers/documents/supplement%205%20-%20multiple%20regression.pdf

LUMS

CENTER FOR SPEECH AND LANGUAGE TECHNOLOGIES

Linear Regression
Overfitting – Bias and Variance

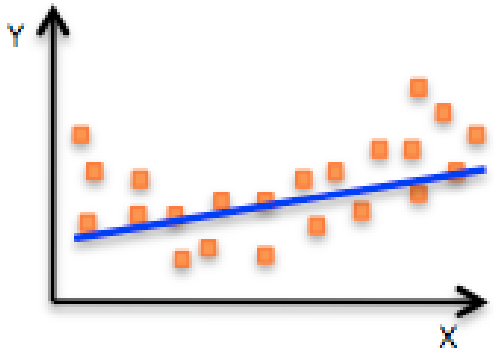
Agha Ali Raza

CS535/EE514 – Machine Learning

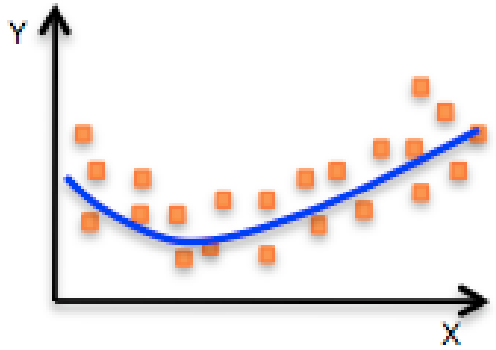
References

- Machine Learning, Andrew Ng, on Coursera by Stanford – a https://www.coursera.org/learn/machine-learning
- Deep Learning Specialization, Andrew Ng, on Coursera by deeplearning.ai – https://www.coursera.org/specializations/deep-learning
- STATQUEST!!! An epic journey through statistics and machine learning, Josh Starmer, https://statquest.org/, https://www.youtube.com/channel/UCtYLUTtgS3k1Fg4y5tAhLbw

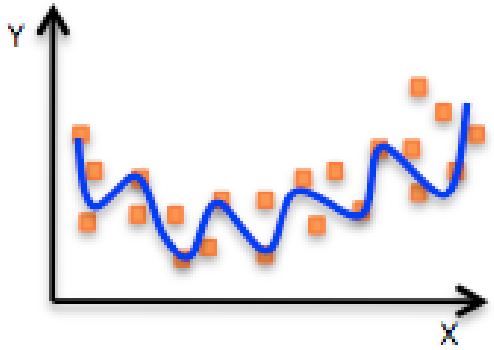
The fitting problem



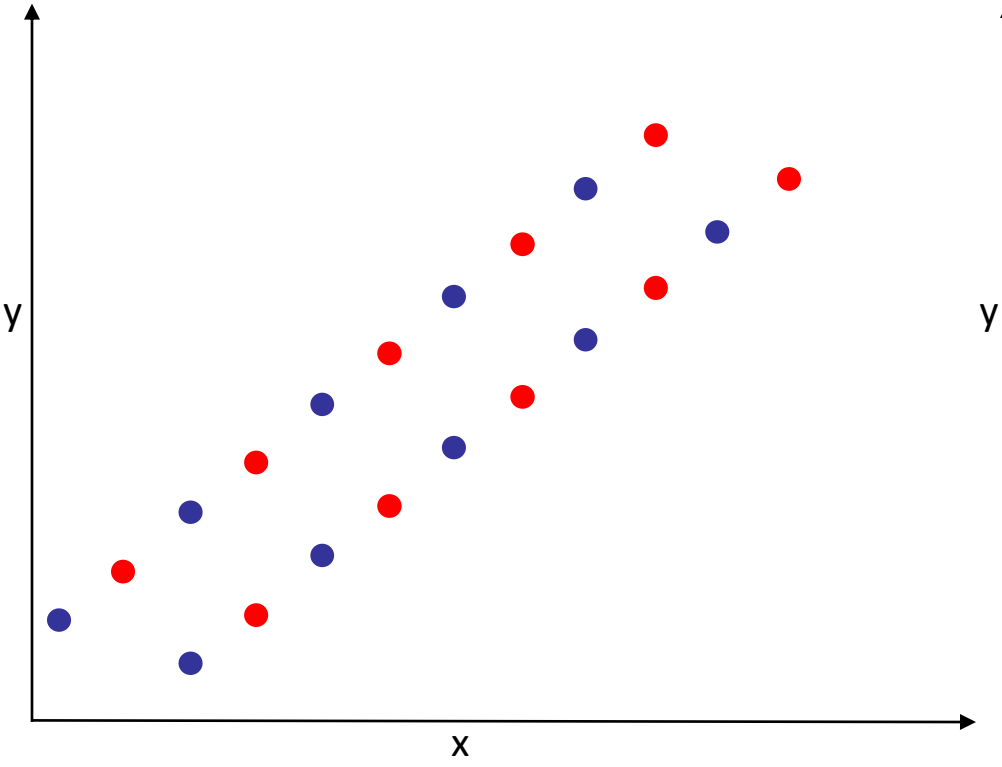
Underfitting

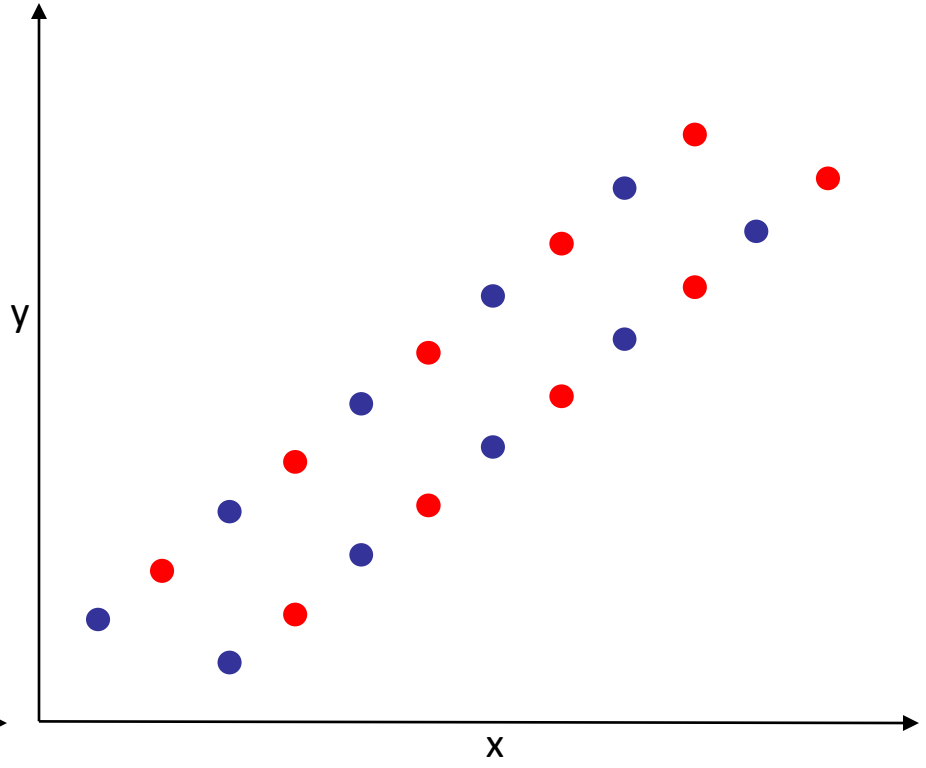


Just right!



overfitting

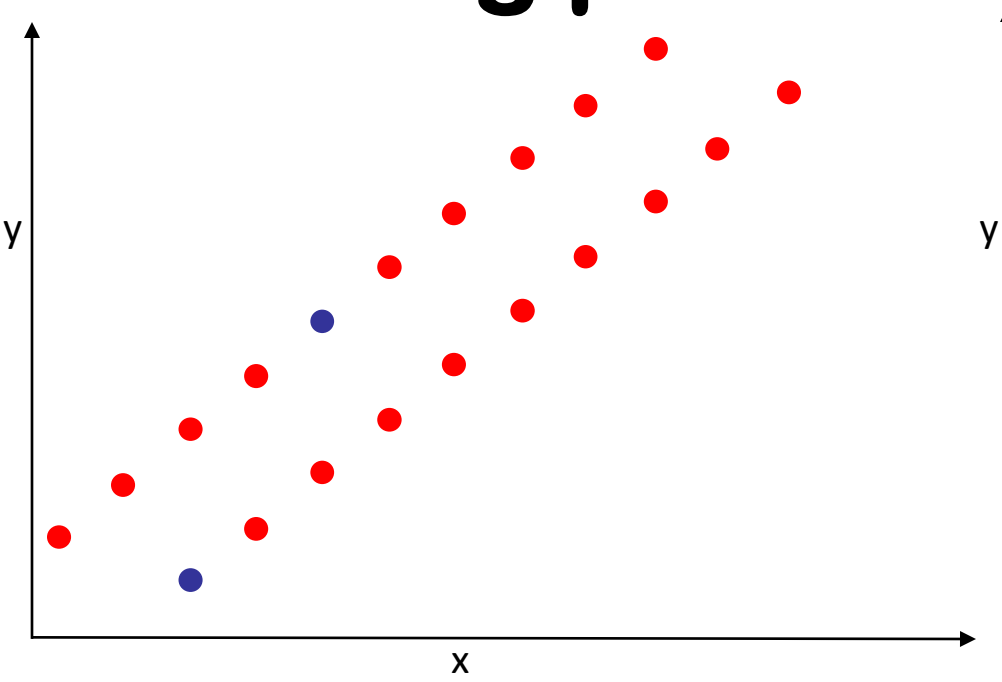


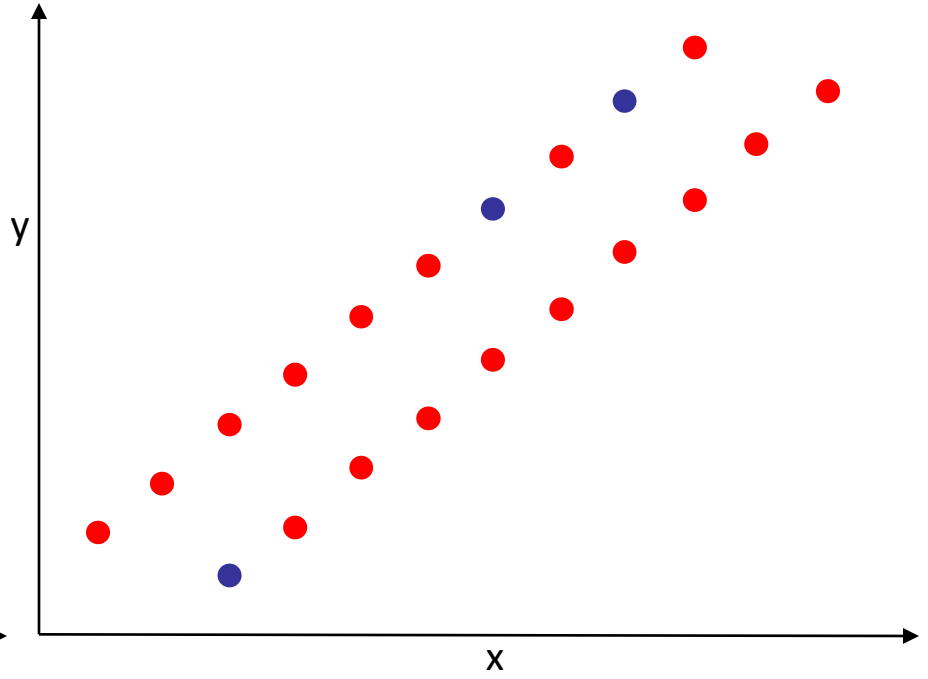


A real example

http://www.petanimalexpo.co.nz/

The fitting problem



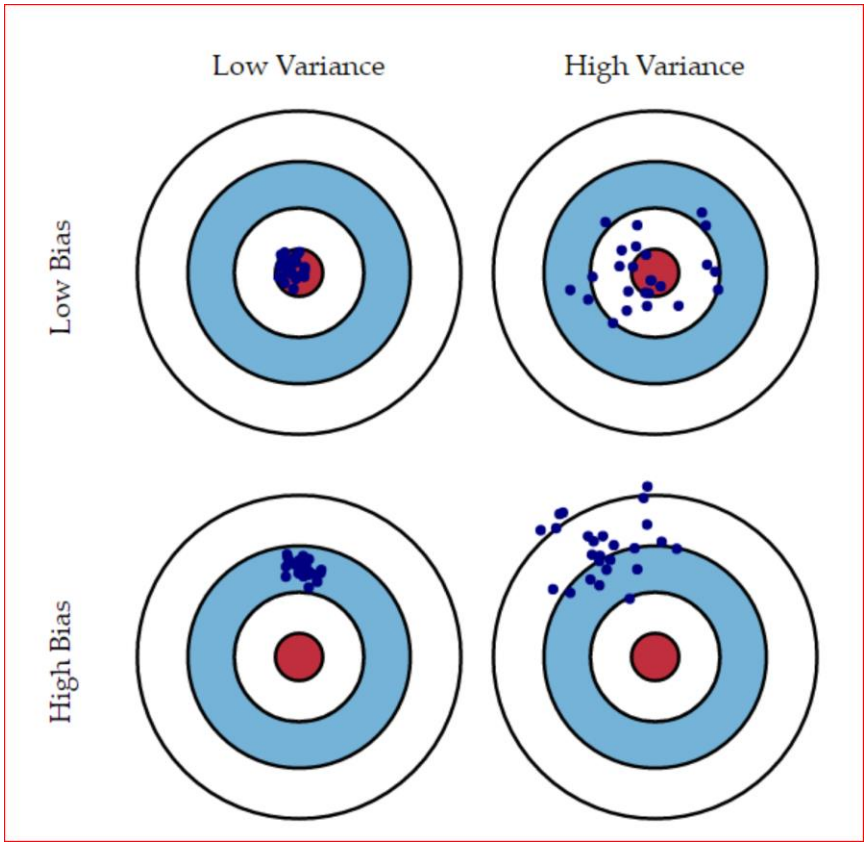


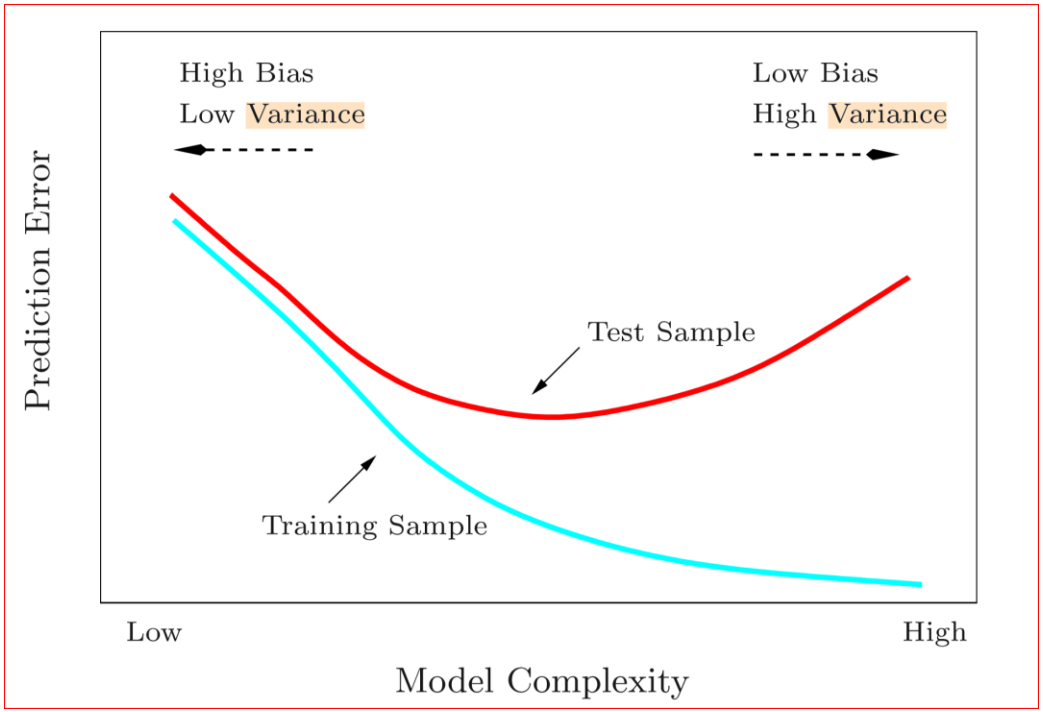
Bias and Variance

- **Bias** is the difference between the average prediction of our model and the correct value which we are trying to predict.
  - If the average predicted values are far off from the actual values, then the bias is high.
  - Model with high bias pays **little attention to the training data** and oversimplifies (presumes a lot about) the model.
  - High bias causes algorithm to **miss relevant relationship between input and output variable**.
  - When a model has a high bias then it implies that the model is too simple and does not capture the complexity of data thus **underfitting** the data.
  - It leads to high error on **training and test data**.
- **Variance** is the variability of model prediction for a given data point or a value which tells us spread of our data. Variance tells us how scattered are the predicted value from the actual value.
  - Model with high variance pays a lot of attention to training data and does not generalize on the data which it hasn't seen before.
  - As a result, such **models perform very well on training data but has high error rates on test data**.
  - High variance causes **overfitting** that implies that the algorithm models random noise present in the training data.

Elements of Statistical Learning by Trevor Hastie, Robert Tibshirani and Jerome Friedman
https://towardsdatascience.com/understanding-the-bias-variance-tradeoff-165e6942b229

Bias and Variance





Elements of Statistical Learning by Trevor Hastie, Robert Tibshirani and Jerome Friedman

https://towardsdatascience.com/understanding-the-bias-variance-tradeoff-165e6942b229

https://medium.com/datadriveninvestor/bias-and-variance-in-machine-learning-51fdd38d1f86

Bias and Variance

Is there a way to find when we have a high bias or a high variance?

- High Bias can be identified when we have
  - High training error
  - Validation error or test error is close to training error
- High Variance can be identified when
  - Low training error
  - High validation error or high test-error

How do we fix high bias or high variance in the data set?

- High bias is due to a simple model and we also see a high training error. To fix that we can do following things:
  - Add more input features
  - Add more complexity by introducing polynomial features
  - Decrease Regularization term
- High variance is due to a model that tries to fit most of the training dataset points and hence gets more complex. To resolve high variance issue we need to work on
  - Getting more training data
  - Reduce input features
  - Increase Regularization term

Elements of Statistical Learning by Trevor Hastie, Robert Tibshirani and Jerome Friedman
https://medium.com/datadriveninvestor/bias-and-variance-in-machine-learning-51fdd38d1f86

Solutions

- Reduce the number of features
  - Manually select features
  - Model selection
- Regularization
  - Reduce magnitude/values of parameters θ_j.
  - Works well when we have a lot of features, each of which contributes a bit to the prediction.
- Bagging and Boosting

LUMS

CENTER FOR SPEECH AND LANGUAGE TECHNOLOGIES

Linear Regression
Manual Feature Selection

Agha Ali Raza

CS535/EE514 – Machine Learning

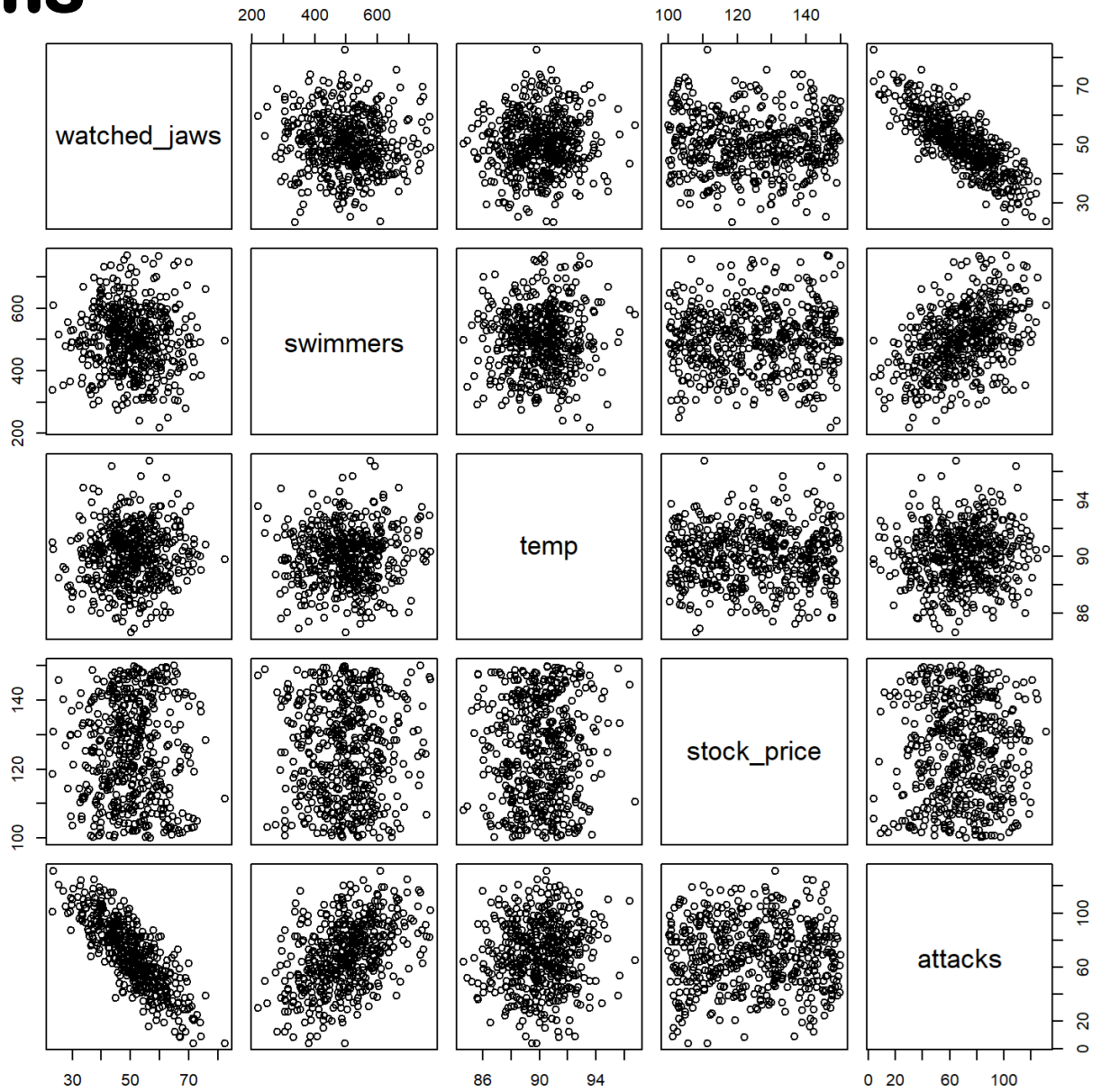
Manually select parameters

Our data frame will consist of 1000 daily measurements of the following independent variables:

- **attacks:** Number of shark attacks (output variable)
- **swimmers:** Number of swimmers in water
- **watched_jaws:** Percentage of swimmers who watched iconic Jaws movies
- **temp:** Average temperature of the day
- **stock_price:** The price of your favorite tech stock that day (a totally unrelated variable)

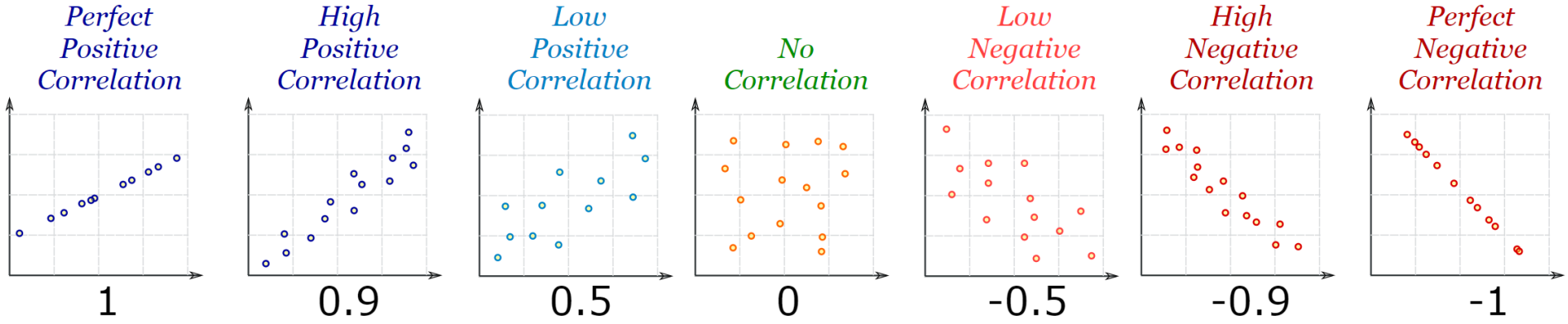
https://scienceloft.com/technical/understanding-lasso-and-ridge-regression/

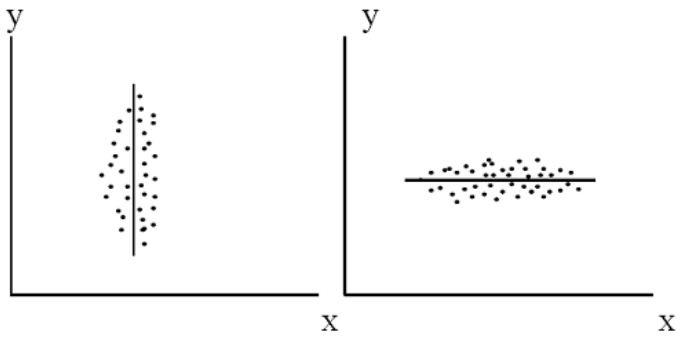
Scatter Diagrams

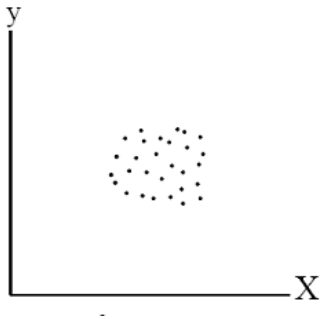


https://scienceloft.com/technical/understanding-lasso-and-ridge-regression/

Scatter Plots



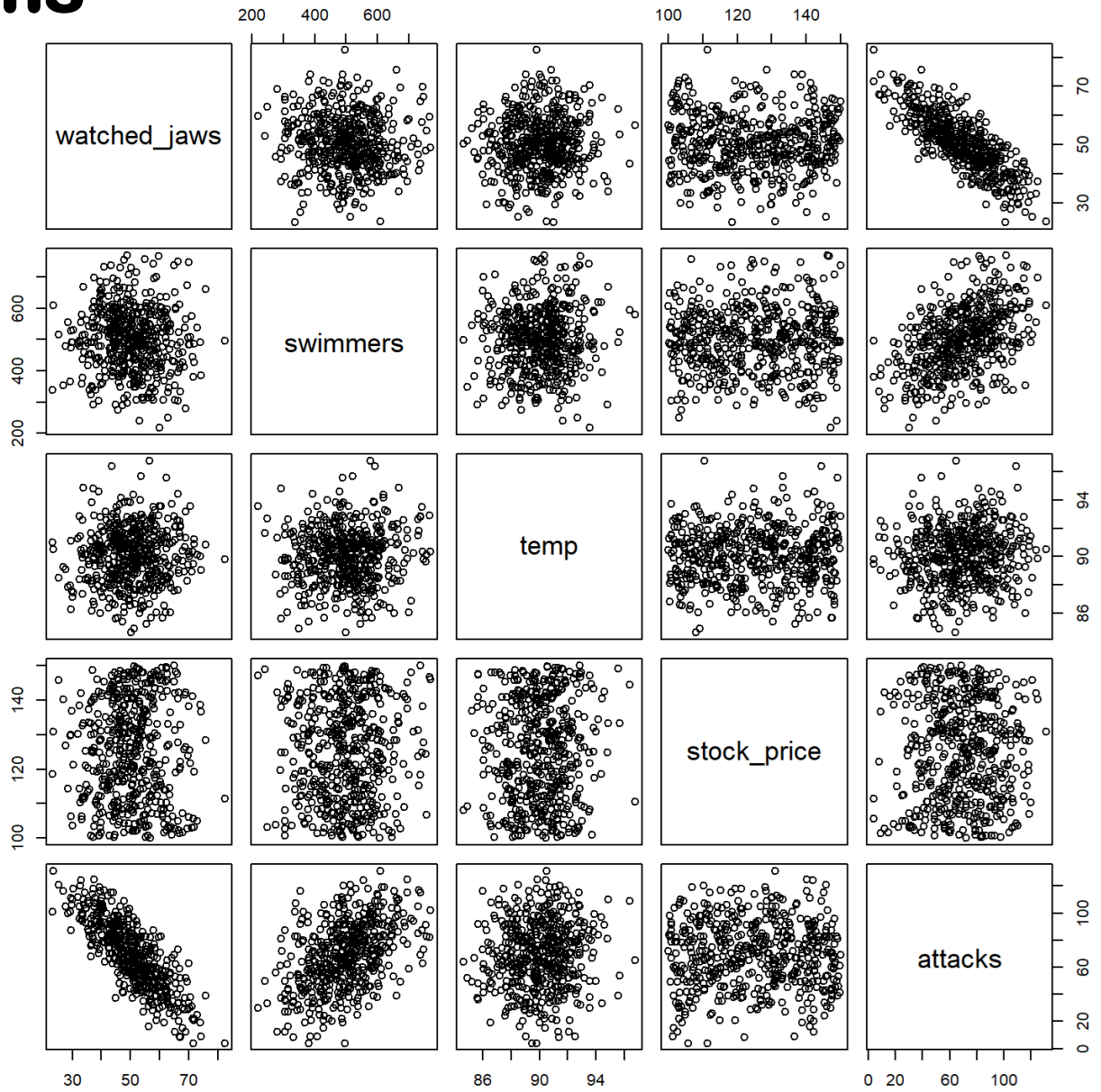




All three of the examples show little to no correlation.

https://www.mathsisfun.com/data/scatter-xy-plots.html, https://study.com/academy/lesson/scatter-plot-and-correlation-definition-example-analysis.html

Scatter Diagrams



https://scienceloft.com/technical/understanding-lasso-and-ridge-regression/

LUMS

CENTER FOR SPEECH AND LANGUAGE TECHNOLOGIES

Linear Regression
Regularization

Agha Ali Raza

CS535/EE514 – Machine Learning

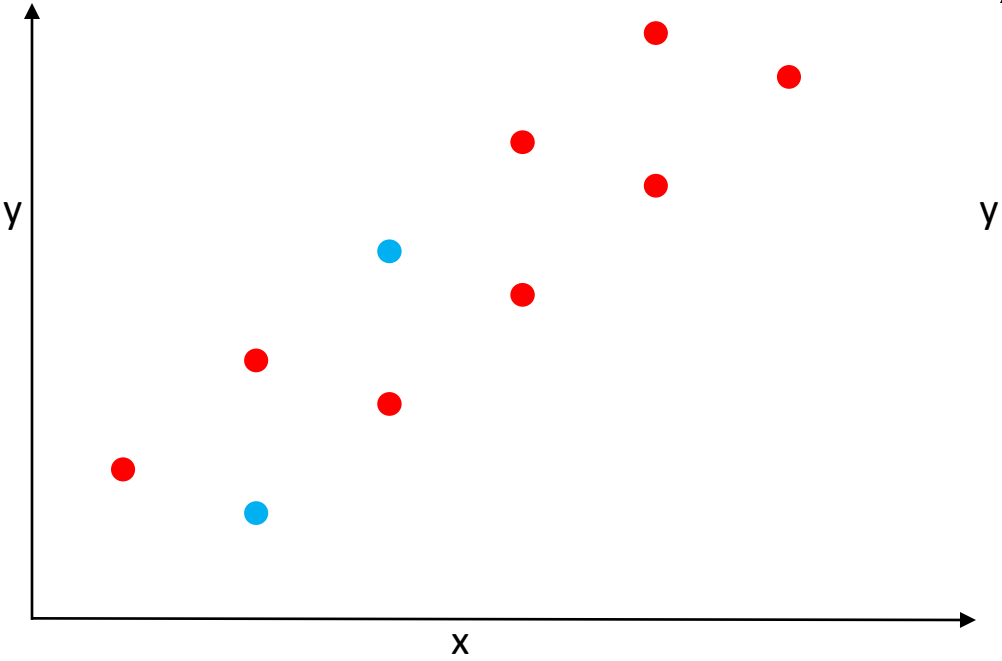
Regularization

- Regularization works on assumption that smaller weights generate simpler model and thus helps avoid overfitting.

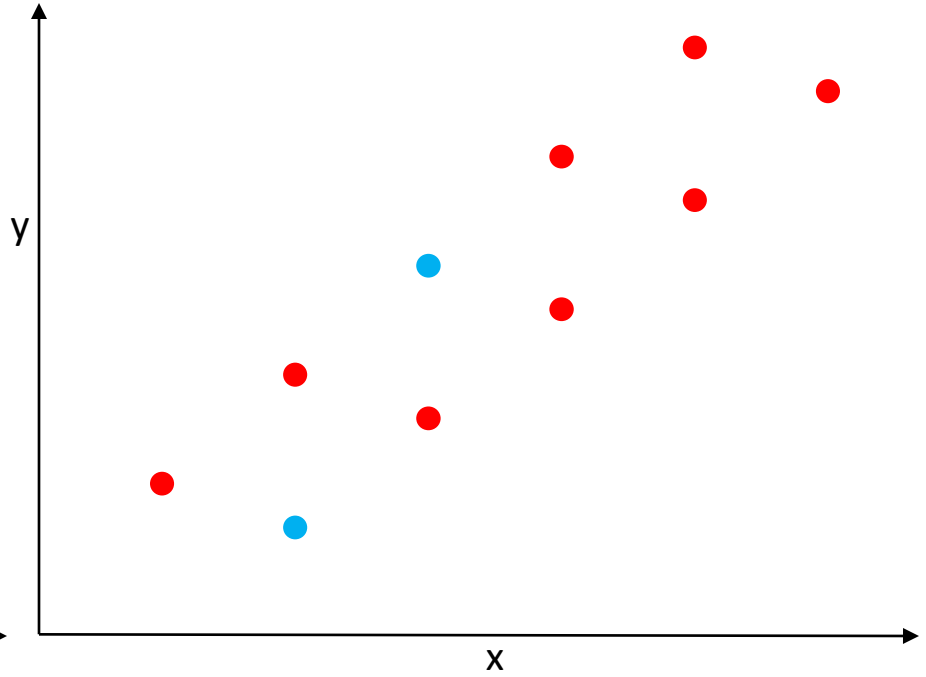
$$f(x_i) = h_\theta x = \theta_0 + \theta_1 x_1 + \theta_2 x_2^2 + \theta_3 x_3^3 + \theta_4 x_4^4$$

$$f(x_i) = h_\theta x = \theta_0 + \theta_1 x_1 + \theta_2 x_2^2$$

The fitting problem



$$h_\Theta(X) = \Theta^T X = \theta_0 + \theta_1 x_1$$



$$h_\Theta(X) = \Theta^T X = \theta_0 + \theta_1 x_1$$

$$J(\Theta) = \frac{1}{2m}\sum_{i=1}^{m}\left(h_\theta(x^{(i)}) - y^{(i)}\right)^2$$

$$\min_\theta \frac{1}{2m}\sum_{i=1}^{m}\left(\theta_0 + \theta_1 x_1^{(i)} - y^{(i)}\right)^2$$

Regularization

$h_\Theta(X) = \theta_0 + \theta_1 x_1 + \cdots + \theta_n x_n$

$$\min_\theta J(\theta) = \min_\theta \frac{1}{2m} \sum_{i=1}^{m} \left(h_\theta(x^{(i)}) - y^{(i)}\right)^2$$

$$\min_\theta J(\theta) = \min_\theta \frac{1}{2m} \left[\sum_{i=1}^{m} \left(h_\theta(x^{(i)}) - y^{(i)}\right)^2 + \lambda \sum_{j=1}^{n} \theta_j^2 \right]$$

$$\min_\theta J(\theta) = \min_\theta \frac{1}{2m} \left[\sum_{i=1}^{m} \left(h_\theta(x^{(i)}) - y^{(i)}\right)^2 + \lambda \sum_{j=1}^{n} |\theta_j| \right]$$

- λ is the penalty term or regularization parameter which determines how much to penalizes the weights
- When λ is zero then the regularization term becomes zero. We are back to the original Loss function
- When λ is large, we penalizes the weights and they become close to zero. This results is a very simple model having a high bias or is underfitting.
  - $h_\Theta(X) = \theta_0$
- Use cross-validation to find the optimal value of λ

https://medium.com/datadriveninvestor/l1-l2-regularization-7f1b4fe948f2

L2 Regularization or Ridge Regression

$$\min_{\theta} J(\theta) = \min_{\theta} \frac{1}{2m}\left[\sum_{i=1}^{m}\left(h_\theta(x^{(i)}) - y^{(i)}\right)^2 + \lambda \sum_{j=1}^{n} \theta_j^2\right]$$

- L2 regularization forces the weights to be small but does not make them zero and does non sparse solution.
- L2 is not robust to outliers as square terms blows up the error differences of the outliers and the regularization term tries to fix it by penalizing the weights
- Ridge regression performs better when all the input features influence the output and all with weights are of roughly equal size

https://medium.com/datadriveninvestor/l1-l2-regularization-7f1b4fe948f2

Y-axis: Regularized coefficients for each variable (i.e. coefficients after penalization is applied)

X-axis: Logarithm of the penalization parameter Lambda. The higher value of lambda indicates more regularization (i.e. reduction of the coefficient magnitude, or shrinkage)

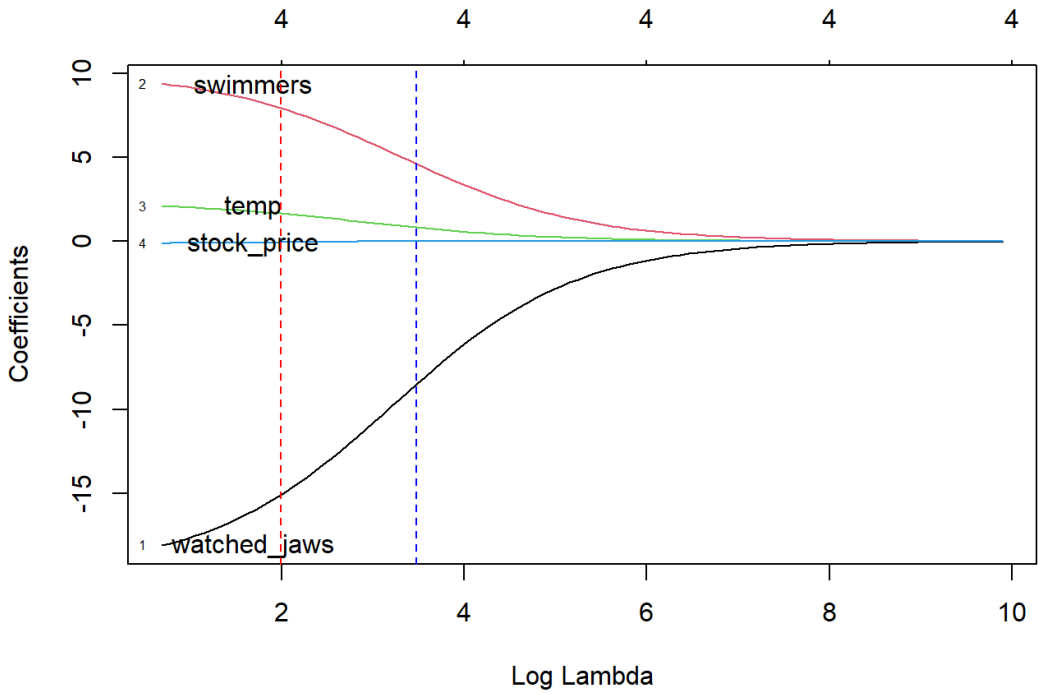
Curves: Change in the predictor coefficients as the penalty term increases.

Numbers on top: The number of variables in the regression model. Since Ridge regression doesn't do feature selection, all the predictors are retained in the final model.

Red dotted line: The minimum value of lambda that results in the smallest cross-validation error.

Blue dotted line: The largest value of lambda within the 1 standard error of the lambda.min. This lambda.1se value corresponds to a higher level of penalization (i.e. more regularized model) and can be chosen for a simpler model in predictions (less impact from coefficients)

Log Lambda = 0 corresponds to "no regularization" (i.e. regular linear model with minimum residual sum of squares).



Watched_jaws has the strongest potential to explain the variation in the output variable, and this remains true as the model regularization increases.
swimmers has the second strongest potential to model the response, but it's importance diminishes near zero as the regularization increases.

https://scienceloft.com/technical/understanding-lasso-and-ridge-regression/

L1 Regularization or Lasso Regression

$$\min_{\theta} J(\theta) = \min_{\theta} \frac{1}{2m} \left[\sum_{i=1}^{m} \left(h_\theta(x^{(i)}) - y^{(i)} \right)^2 + \lambda \sum_{j=1}^{n} |\theta_j| \right]$$

- L1 norm shrinks the parameters to zero.
  - When input features have weights closer to zero that leads to sparse L1 norm. In Sparse solution majority of the input features have zero weights and very few features have non-zero weights.
- Not all input features have the same influence on the prediction. L1 norm will assign a zero weight to features with less predictive power.
- L1 regularization does feature selection. It does this by assigning insignificant input features with zero weight and useful features with a non-zero weight.

https://medium.com/datadriveninvestor/l1-l2-regularization-7f1b4fe948f2

Y-axis: Regularized coefficients for each variable (i.e. coefficients after penalization is applied)

X-axis: Logarithm of the penalization parameter Lambda. The higher value of lambda indicates more regularization (i.e. reduction of the coefficient magnitude, or shrinkage)

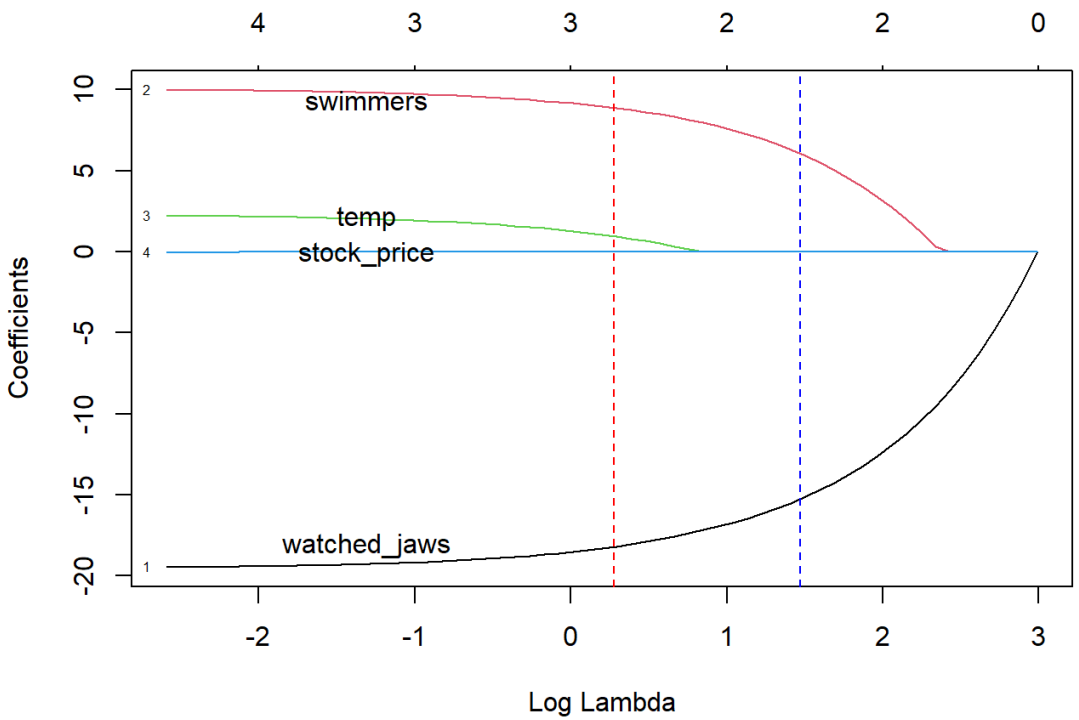
Curves: Change in the predictor coefficients as the penalty term increases.

Numbers on top: The number of variables in the regression model. Since Ridge regression doesn't do feature selection, all the predictors are retained in the final model.

Red dotted line: The minimum value of lambda that results in the smallest cross-validation error.

Blue dotted line: The largest value of lambda within the 1 standard error of the lambda.min. This lambda.1se value corresponds to a higher level of penalization (i.e. more regularized model) and can be chosen for a simpler model in predictions (less impact from coefficients)

Log Lambda = 0 corresponds to "no regularization" (i.e. regular linear model with minimum residual sum of squares).



temp and stock_price get eliminated quickly.

https://scienceloft.com/technical/understanding-lasso-and-ridge-regression/

Differences

L1 Regularization

- L1 penalizes sum of absolute value of weights.
- L1 has a sparse solution
- L1 has multiple solutions
- L1 has built in feature selection
- L1 is robust to outliers
- L1 generates model that are simple and interpretable but cannot learn complex patterns

L2 Regularization

- L2 regularization penalizes sum of square weights.
- L2 has a non sparse solution
- L2 has one solution
- L2 has no feature selection
- L2 is not robust to outliers
- L2 gives better prediction when output variable is a function of all input features
- L2 regularization is able to learn complex data patterns

Elastic net regularization is a combination of both L1 and L2 regularization

$$\min_{\theta} J(\theta) = \min_{\theta} \frac{1}{2m}\left[\sum_{i=1}^{m}\left(h_\theta(x^{(i)}) - y^{(i)}\right)^2 + \lambda_1 \sum_{j=1}^{n}|\theta_j| + \lambda_2 \sum_{j=1}^{n}\theta_j^2\right]$$

https://medium.com/datadriveninvestor/l1-l2-regularization-7f1b4fe948f2

For more details please visit

http://aghaaliraza.com

Thank you!